

✓ Chapter 3 - Diffusion Models

This chapter is a **gentle introduction** to diffusion models (mostly **DDMP**)

By the end of this chapter, you should be able to:

1. Review what is a **Markov Chain**
2. Understand the basic idea behind **Diffusion** generative processes
3. Know the various **families** of **Diffusion** processes
4. Understand the **forward** diffusion process
5. Explore why modeling the **reverse** process is hard and how to get around this problem
6. Run simple **examples** of **forward** and **backwards** processes
7. Understand how a **neural network** needs to be trained for the **backwards process**
8. Understand why **sinoidal encoding** is required for **time** input to the **neural network**
9. Understand why the **UNET** is an appropriate **architecture** for **Diffusion** processes

License

Text, figures, and explanations:

© 2026 Imran Zualkernan. Licensed under **CC BY 4.0**.

Code cells:

© 2026 Imran Zualkernan. Licensed under the **MIT License**.

You are free to reuse, modify, and redistribute with attribution.

Review: What Is a Markov Chain?

A **Markov chain** is a sequence of random variables

$$x_0, x_1, x_2, \dots$$

that satisfies the **Markov property**:

$$p(x_t \mid x_{t-1}, x_{t-2}, \dots, x_0) = p(x_t \mid x_{t-1}).$$

In words:

The future depends only on the present, not on the past.

The current state x_{t-1} contains all the information needed to predict x_t .

Transition Distribution

A Markov chain is fully specified by:

1. An initial distribution:

$$p(x_0)$$

2. A transition distribution:

$$p(x_t \mid x_{t-1})$$

The joint distribution factorizes as:

$$p(x_{0:T}) = p(x_0) \prod_{t=1}^T p(x_t \mid x_{t-1}).$$

This factorization is what makes Markov chains tractable.

Key Properties

Local Dependence

Each step depends only on the previous step:

$$x_{t-1} \rightarrow x_t \rightarrow x_{t+1}.$$

Graphically:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

Marginalization

Even though the joint distribution involves all states,

$$p(x_t) = \int p(x_{0:t}) dx_0 \dots dx_{t-1}.$$

Forward and Reverse Chains

If the forward process is Markov:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1}),$$

then under mild conditions, a reverse Markov chain also exists:

$$q(x_{0:T}) = q(x_T) \prod_{t=1}^T q(x_{t-1} \mid x_t).$$

Basic Idea Behind Diffusion Models

The goal of a diffusion model is to **start from a simple distribution** (typically standard Gaussian noise) and generate samples that look like data from a target distribution. In other words, we want a procedure that takes

$$x_T \sim \mathcal{N}(0, I)$$

and produces a final sample x_0 whose distribution matches the data

$$x_0 \sim q_{\text{data}}(x).$$

Diffusion models achieve this by learning to **reverse** a carefully designed *forward noising process*.

Step 1 — Define a Forward Process (Data $\rightarrow$ Gaussian)

We construct a Markov chain: \$\$

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

\$\$ where:

- $x_0 \sim q_{\text{data}}$ (real data)
- Each step adds a small amount of Gaussian noise
- After many steps, $x_T \approx \mathcal{N}(0, I)$

This direction is easy to define and simulate. It requires **no learning**.

Step 2 — Learn the Reverse Process (Gaussian $\rightarrow$ Data)

Probability theory guarantees that a reverse Markov chain exists:

\$\$

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.$$

\$\$

If we could model this reverse process perfectly, then starting from

\$\$

$x_T \sim \mathcal{N}(0, I)$

\$\$

and running the reverse chain would generate samples from the true data distribution.

Why This Makes Sense as a Generative Model

Instead of directly learning a mapping

$$\mathcal{N}(0, I) \rightarrow \text{data}$$

(which is very difficult), diffusion breaks the problem into many small steps:

- Each reverse step only needs to remove **a small amount of noise**
- Each step is local and approximately Gaussian
- The learning problem becomes stable and well-conditioned

Summary

Adding Gaussian noise progressively:

- Increases entropy
- Smooths the data distribution
- Pushes everything toward a simple Gaussian

The reverse model learns the local direction that moves noisy points back toward regions of high data density. Generation is therefore reversing a known stochastic process. Diffusion models work because we define a tractable noise process that maps data to Gaussian noise, and then learn to reverse it step-by-step to generate new data.

Diffusion models: the main families

The term "**diffusion model**" now covers several closely-related families. We will primarily focus on **DDPM-style Gaussian diffusion** (discrete-time), but the broader landscape is given in table below.

A table of common diffusion families

Family / Name	Forward corruption	Wha
DDPM (Denoising Diffusion Probabilistic Models)	Gaussian Markov chain (VP)	$\epsilon_\theta(x_t, t)$ or μ_θ
DDIM (Implicit diffusion)	Same as DDPM	Same network
Score-SDE (continuous-time score-based)	SDE (VP / VE / sub-VP)	score $s_\theta(x, t)$
VE vs VP schedules	Gaussian corruption	Same as above
Latent Diffusion (LDM)	Diffuse in latent space	$\epsilon_\theta(z_t, t, c)$
Guided diffusion	Same forward	Add guidance
Consistency / Distillation	Usually trained from diffusion teacher	direct map x_t
Rectified flows / Flow matching (diffusion-adjacent)	Often deterministic path	vector field v_θ
Discrete diffusion (categorical)	Masking / discrete corruption	logits / denois

✓ Notation

It's easy to get lost in notation like $p_\theta(x_{0:T})$ or $q(x_{1:T} \mid x_0)$. This notation is extremely important to understand correctly.

What does $x_{1:T}$ mean?

It is **shorthand** for the sequence:

$$x_{1:T} = (x_1, x_2, x_3, \dots, x_T).$$

It does **not** mean:

- prediction,
- exponentiation,
- multiplication,
- or "x at stage T."

It simply means **all variables from time step 1 through time step T**.

If $T = 3$, then:

$$x_{1:3} = (x_1, x_2, x_3).$$

What does $q(x_{1:T} \mid x_0)$ mean?

It means:

\$\$

$$q(x_{1:T} \mid x_0)$$

$$q(x_1, x_2, \dots, x_T \mid x_0).$$

\$\$

This is the **joint probability density of the entire noisy trajectory**, given the starting data point x_0 .

What is a “trajectory”?

A trajectory is the full sequence: $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$.

\$\$

In the **forward process**, we generate this trajectory by repeatedly adding noise.

So a trajectory is simply:

All intermediate noisy versions of the same original data point.

Are we “predicting the whole sequence from x_0 ”?

Not exactly.

In the **forward process**, the trajectory is sampled using fixed Gaussian transitions. We are not predicting it, we are sampling it.

In the **reverse process**, the model learns:

\$\$

$$p_{\theta}(x_{t-1} \mid x_t)$$

\$\$

So during generation we:

1. Sample $x_T \sim \mathcal{N}(0, 1)$
2. Predict x_{T-1} from x_T
3. Predict x_{T-2} from x_{T-1}
4. Continue until x_0

In that sense, during sampling, we are generating the full reverse trajectory step-by-step.

Trajectory vs. Marginal

There are two different objects:

Joint trajectory:

$$q(x_{1:T} \mid x_0)$$

\$\$

Probability of the entire noisy path.

Single marginal:

\$\$

$$q(x_t \mid x_0)$$

\$\$

Probability of just the state at time t (after integrating out the others).

Think of diffusion as a random walk in time.

We start at:

$$x_0$$

and then generate a random path:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T.$$

Because each step is stochastic, there are **many possible paths**.

Many Possible Paths

For a fixed x_0 , the forward process generates a tree of possibilities. Each path corresponds to one realization of the Gaussian noises:

$$\epsilon_1, \epsilon_2, \dots, \epsilon_t.$$

The Joint Trajectory

The joint distribution

$$q(x_{1:T} \mid x_0)$$

assigns probability to a **specific full path**:

$$(x_1, x_2, \dots, x_T).$$

It tells us:

What is the probability of this exact sequence of noisy states?

The Marginal at Time t

The marginal

$$q(x_t \mid x_0)$$

does not care about the path.

It asks:

After t steps of noise, where could we be?

Mathematically, we integrate over all possible intermediate paths:

$$q(x_t \mid x_0) = \int q(x_{1:t} \mid x_0) dx_1 \dots dx_{t-1}.$$

So we are summing over all paths that end at x_t .

Why Do We Integrate Over Intermediate Paths?

We want the probability of being at state x_t given the start x_0 :

$$q(x_t \mid x_0).$$

But the forward process defines a *joint distribution* over the entire trajectory:

$$q(x_{1:t} \mid x_0) = q(x_1, x_2, \dots, x_t \mid x_0).$$

This distribution assigns probability to complete paths.

We are using the Law of Marginalization

In probability theory, if we have a joint distribution:

$$p(a, b),$$

then the marginal over a is:

$$p(a) = \int p(a, b) db.$$

We are simply applying the same rule here.

Since

$$q(x_{1:t} \mid x_0) = q(x_1, \dots, x_{t-1}, x_t \mid x_0),$$

the marginal of x_t is obtained by integrating out $x_1, \dots, x_{t-1}$:

$$q(x_t \mid x_0) = \int q(x_1, \dots, x_{t-1}, x_t \mid x_0) dx_1 \dots dx_{t-1}.$$

What Does This Mean Intuitively?

Each possible sequence

$$(x_1, x_2, \dots, x_{t-1})$$

defines one possible **path** that leads from x_0 to x_t . The joint distribution assigns probability to each such path.

To compute the probability of just x_t , we must:

Add up the probability of *all possible paths* that end at x_t .

That is exactly what the integral does.

Summary

- The trajectory describes randomness over time.
- The marginal describes uncertainty at a fixed time.
- Diffusion training only needs the marginal.
- Sampling uses the full reverse trajectory.

✓ Forward and Reverse Transitions - Core Markov Structure

At the heart of diffusion models is a **Markov chain**.

Forward (Noising) Transition

The forward diffusion process is defined step-by-step as:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t))$$

Equivalently, sampling can be written as:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

This defines the full forward chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1}).$$

Each step slightly shrinks the signal and injects Gaussian noise. After many steps, the distribution approaches:

$$x_T \approx \mathcal{N}(0, I).$$

Reverse (Denoising) Transition

The corresponding reverse transition is:

$$q(x_{t-1} \mid x_t).$$

This is the true backward transition of the forward Markov chain. If we could compute it exactly and sample from:

$$x_T \sim \mathcal{N}(0, I),$$

then repeatedly applying

$$q(x_{t-1} \mid x_t)$$

would generate samples from the data distribution.

Why the Reverse Is Hard

While the forward transition is explicitly defined and tractable, the reverse transition

$$q(x_{t-1} \mid x_t)$$

requires averaging over all possible original clean data points that could have produced x_t . This is intractable in practice and therefore, diffusion models introduce a neural network to approximate the reverse step:

$$p_\theta(x_{t-1} \mid x_t) \approx q(x_{t-1} \mid x_t).$$

Learning this reverse process is what makes diffusion a generative model.

Why does $q(x_t \mid x_{t-1})$ look like this and why?

In DDPM, the forward transition is defined as:

$$q(x_t \mid x_{t-1})$$

$$\mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t))$$

\$\$

This means:

- It is a **Gaussian distribution**
- Mean = $\sqrt{\alpha_t} x_{t-1}$
- Variance = $1 - \alpha_t$

We define the forward diffusion step as a Gaussian conditional:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \mu_t, \sigma_t^2).$$

In DDPM we choose:

$$\mu_t = \sqrt{\alpha_t} x_{t-1}, \quad \sigma_t^2 = 1 - \alpha_t.$$

Therefore,

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, 1 - \alpha_t).$$

From properties of the Normal distribution:

- Mean:

$$\mathbb{E}[x_t \mid x_{t-1}] = \sqrt{\alpha_t} x_{t-1}$$

- Variance:

$$\text{Var}(x_t \mid x_{t-1}) = 1 - \alpha_t$$

We know from the reparameterization trick that:

If

$$X \sim \mathcal{N}(\mu, \sigma^2),$$

then it can be written as:

$$X = \mu + \sigma \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

Applying this idea this to the forward transition with:

$$\mu = \sqrt{\alpha_t} x_{t-1}, \quad \sigma = \sqrt{1 - \alpha_t}.$$

Then we obtain the sampling equation:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

We can clearly see that each step does two things:

1. **Shrink toward zero** $\sqrt{\alpha_t} < 1$ scales the previous value x_{t-1} down slightly.
2. **Inject fresh Gaussian noise** $\sqrt{1 - \alpha_t} \varepsilon$ adds randomness.

Repeated many times, this gradually destroys structure and pushes the distribution toward a standard normal.

Why this specific Gaussian form?

This choice gives three crucial properties:

Markov structure

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1})$$

Each step depends only on the previous one.

Closed-form marginal

Because the transition is linear Gaussian, we can compute:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, 1 - \bar{\alpha}_t)$$

where:

\$\$

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

This closed form makes training tractable.

Proof: closed-form marginal $q(x_t \mid x_0)$

Assume the DDPM forward step is

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, 1 - \alpha_t),$$

equivalently

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1)$$

with $\varepsilon_1, \dots, \varepsilon_t$ independent.

Define

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

We prove by induction that

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, 1 - \bar{\alpha}_t).$$

Base case $t = 1$

From the definition of the forward step:

$$q(x_1 \mid x_0) = \mathcal{N}(x_1; \sqrt{\alpha_1} x_0, 1 - \alpha_1).$$

Since $\bar{\alpha}_1 = \alpha_1$, this matches

$$\mathcal{N}(x_1; \sqrt{\bar{\alpha}_1} x_0, 1 - \bar{\alpha}_1).$$

So the claim holds for $t = 1$.

Inductive hypothesis

Assume for some $t - 1 \geq 1$ that

$$x_{t-1} \mid x_0 \sim \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}} x_0, 1 - \bar{\alpha}_{t-1}).$$

One-step propagation

From the forward update:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1),$$

and ε_t is independent of x_{t-1} (hence independent of x_0 given x_{t-1}).

A linear transformation of a Gaussian plus independent Gaussian noise is Gaussian, so $x_t \mid x_0$ is Gaussian. We compute its mean and variance.

Mean:

$$\mathbb{E}[x_t \mid x_0] = \mathbb{E}[\sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \mid x_0] = \sqrt{\alpha_t} \mathbb{E}[x_{t-1} \mid x_0] + \sqrt{1 - \alpha_t} \mathbb{E}[\varepsilon_t \mid x_0]$$

Since $\mathbb{E}[\varepsilon_t] = 0$ and by the inductive hypothesis $\mathbb{E}[x_{t-1} \mid x_0] = \sqrt{\bar{\alpha}_{t-1}} x_0$, we get:

$$\mathbb{E}[x_t \mid x_0] = \sqrt{\alpha_t} \sqrt{\bar{\alpha}_{t-1}} x_0 = \sqrt{\alpha_t \bar{\alpha}_{t-1}} x_0 = \sqrt{\bar{\alpha}_t} x_0.$$

Variance:

Using independence of x_{t-1} and ε_t given x_0 :

$$\text{Var}(x_t \mid x_0) = \text{Var}(\sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \mid x_0) = \alpha_t \text{Var}(x_{t-1} \mid x_0) + (1 - \alpha_t) \text{Var}(\varepsilon_t \mid x_0)$$

Since $\text{Var}(\varepsilon_t) = 1$ and $\text{Var}(x_{t-1} \mid x_0) = 1 - \bar{\alpha}_{t-1}$ by the inductive hypothesis:

$$\text{Var}(x_t \mid x_0) = \alpha_t(1 - \bar{\alpha}_{t-1}) + (1 - \alpha_t) = \alpha_t - \alpha_t \bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \alpha_t \bar{\alpha}_{t-1} =$$

We have shown $x_t \mid x_0$ is Gaussian with mean $\sqrt{\bar{\alpha}_t} x_0$ and variance $1 - \bar{\alpha}_t$:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, 1 - \bar{\alpha}_t),$$

where $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

This completes the induction.

Convergence to Gaussian noise

As $t \rightarrow T$, we have $\bar{\alpha}_t \rightarrow 0$, so:

$$q(x_T \mid x_0) \approx \mathcal{N}(0, 1)$$

This allows reverse sampling to start from:

$$p(x_T) = \mathcal{N}(0, 1)$$

Geometric interpretation

Each forward step:

- Contracts the signal
- Increases entropy
- Smooths the distribution

It is equivalent to repeatedly applying a small Gaussian blur in latent space.

The Role of α_t , β_t , and the Noise Schedule

In diffusion models, the forward process is defined as:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, 1 - \alpha_t).$$

Equivalently, introducing

$$\beta_t := 1 - \alpha_t,$$

we can write:

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

What Do α_t and β_t Mean?

- α_t controls **how much signal remains** at step t .
- β_t controls **how much new noise is injected** at step t .

They are related by:

$$\alpha_t = 1 - \beta_t.$$

Conceptually:

- β_t is the variance of newly added noise.
- $\sqrt{\alpha_t}$ shrinks the previous signal.
- Small $\beta_t \rightarrow$ slow corruption.
- Large $\beta_t \rightarrow$ fast corruption.

Cumulative Signal Strength

Define the cumulative product:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Then we can compute in closed form:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, 1 - \bar{\alpha}_t).$$

Interpretation:

- $\sqrt{\bar{\alpha}_t}$ tells us how much of the original signal survives.
- $1 - \bar{\alpha}_t$ tells us how much total noise has accumulated.

To ensure complete corruption, we choose the schedule such that:

$$\bar{\alpha}_T \approx 0,$$

so that:

$$x_T \approx \mathcal{N}(0, I).$$

Why Introduce β_t ?

We introduce β_t because:

- It directly represents **noise variance**.
- It is easier to design a schedule for injected noise.
- It aligns with the continuous-time SDE formulation (later).
- It simplifies KL and ELBO derivations (later).

Diffusion is fundamentally about **gradually adding Gaussian noise**, so variance is the natural control parameter.

Common Time Schedules

The sequence $\{\beta_t\}_{t=1}^T$ is called the **noise schedule**.

Linear Schedule (Original DDPM)

β_t increases linearly from $\beta_{\min}$ to $\beta_{\max}$.

Typical values:

- $\beta_{\min} = 10^{-4}$
- $\beta_{\max} = 0.02$

Simple and effective, but not optimal for SNR decay.

Cosine Schedule (Improved DDPM)

Instead of defining β_t directly, define:

$$\bar{\alpha}_t = \frac{\cos^2\left(\frac{t/T+s}{1+s} \frac{\pi}{2}\right)}{\cos^2\left(\frac{s}{1+s} \frac{\pi}{2}\right)}.$$

Then compute:

$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}.$$

This gives smoother signal-to-noise decay and better sample quality.

Signal-to-Noise Ratio (SNR)

Define:

$$\text{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}.$$

The schedule determines how quickly SNR decays:

- Slow decay $\rightarrow$ easier denoising early.
- Fast decay $\rightarrow$ more aggressive corruption.

Good schedules balance learning difficulty across timesteps.

Summary

- β_t = injected noise variance (design choice).
- $\alpha_t = 1 - \beta_t$ = remaining signal fraction.
- $\bar{\alpha}_t$ = cumulative signal strength.
- The schedule controls:
 - Information decay
 - Training stability

- Sample quality
- Reverse process difficulty

The entire behavior of diffusion is governed by the noise schedule.

✓ Simple Example of the forward diffusion process

```
import math
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

torch.manual_seed(0)
np.random.seed(0)

device = "cuda" if torch.cuda.is_available() else "cpu"
```

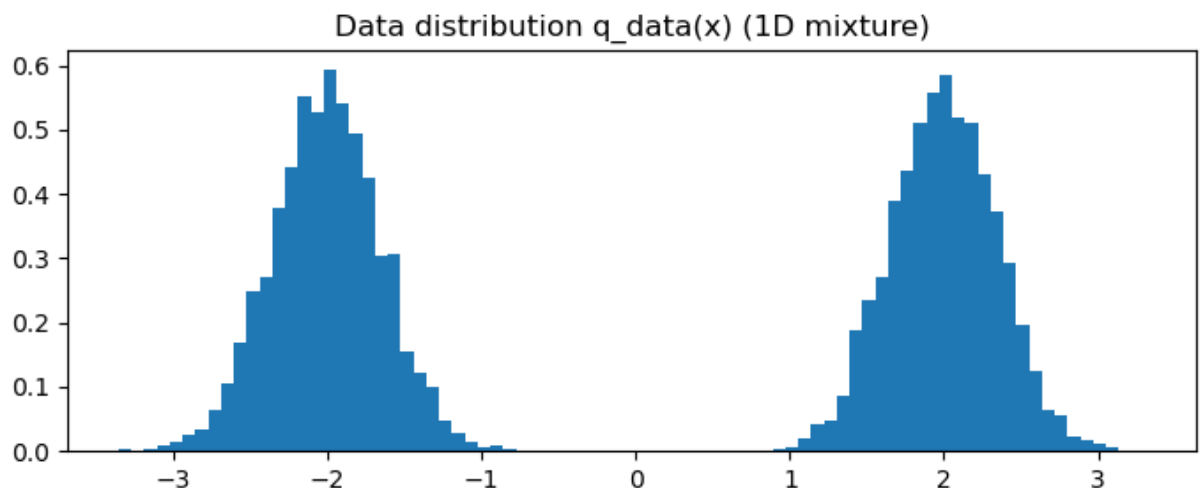
✓ Data to be sampled from

We create a 1D mixture distribution as the target data set.

```
def sample_1d_mixture(n=8000, means=(-2.0, 2.0), sigma_data=0.35, w
    means_t = torch.tensor(means, device=device).float()
    weights_t = torch.tensor(weights, device=device).float()
    comp = torch.multinomial(weights_t, n, replacement=True)
    x = means_t[comp] + sigma_data * torch.randn(n, device=device)
    return x.view(-1, 1), comp

x0, comp = sample_1d_mixture(device=device)

plt.figure(figsize=(7,3))
plt.hist(x0.detach().cpu().numpy().ravel(), bins=80, density=True)
plt.title("Data distribution q_data(x) (1D mixture)")
plt.tight_layout()
plt.show()
```



✓ Defining the forward diffusion process for the 1D data

```
# Using a linear beta schedule for simplicity, but many other sched
# The betas are the noise variances at each step, and they control
# We calculate alpha bar values here for convenience, as they are u

def make_linear_beta_schedule(T, beta_start=1e-4, beta_end=2e-2, de
    betas = torch.linspace(beta_start, beta_end, T, device=device)
    alphas = 1.0 - betas
    alphaBars = torch.cumprod(alphas, dim=0)
    return betas, alphas, alphaBars

# We define the total number of steps to noise
```

$T = 200$

```
# for each t, we will get a beta, alpha, and alpha_bar value that c
betas, alphas, alpha_bars = make_linear_beta_schedule(T, device=dev
```

```
# This function samples from the forward diffusion process  $q(x_t |$ 
# This is the marginal distribution of  $x_t$ , which is a Gaussian wit
# For any  $x_0$  and  $t$ , it allows us to sample a noisy version of  $x_0$  at
```

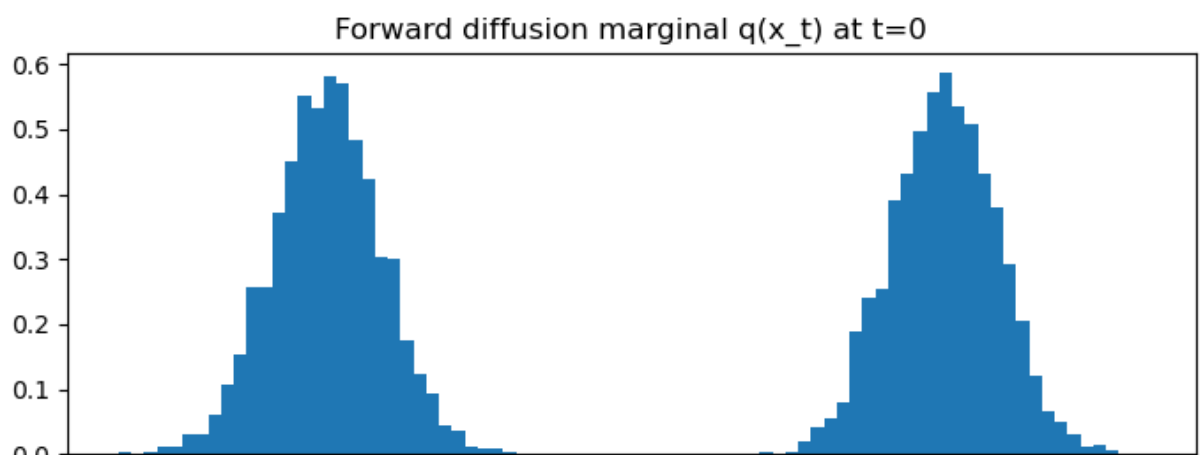
```
def q_sample(x0, t, alpha_bars, noise=None):
    if noise is None:
        noise = torch.randn_like(x0)
    # view alpha_bar as a column vector so it can be broadcasted wi
    a_bar = alpha_bars[t].view(-1, 1)

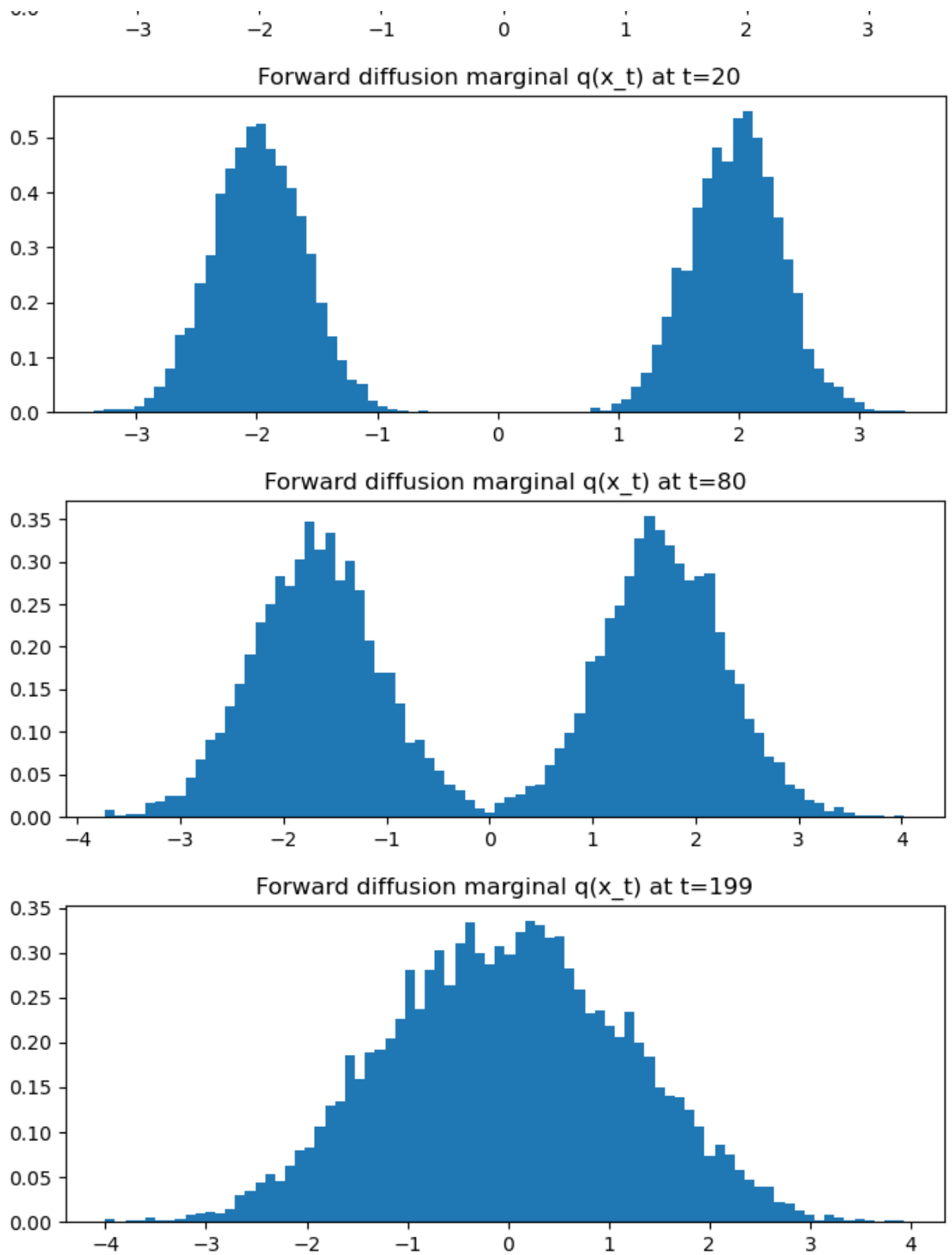
    # the formula for  $q(x_t | x_0)$  is derived from the fact that th
    # Gaussian transitions, and we can compute the mean and varianc
    xt = torch.sqrt(a_bar) * x0 + torch.sqrt(1.0 - a_bar) * noise
    return xt, noise
```

```
# This loop visualizes the distribution of  $x_t$  at different time st
# As  $t$  increases, we expect the distribution to become more Gaussia
```

```
for t_scalar in [0, 20, 80, 199]:
```

```
    # tt is a tensor of shape (batch_size,) where each entry is the
    # This allows us to sample a batch of  $x_t$  values at the same ti
    tt = torch.full((x0.shape[0],), t_scalar, dtype=torch.long, dev
    # sample  $x_t$  from the forward process given  $x_0$  and  $t$ , and plot
    xt, _ = q_sample(x0, tt, alpha_bars)
    plt.figure(figsize=(7,3))
    plt.hist(xt.detach().cpu().numpy().ravel(), bins=80, density=True)
    plt.title(f"Forward diffusion marginal  $q(x_t)$  at  $t=\{t\_scalar\}$ ")
    plt.tight_layout()
    plt.show()
```





✓ We can visualize how the forward process morphs the original distribution into $N(0,1)$

```
# Geometric visualization: forward diffusion as Gaussian smoothing (
#
# In 1D,  $q(x_t)$  can be viewed as the data density  $q_{\text{data}}$  "blurred" b
# (plus a scaling of  $x_0$  by  $\sqrt{\alpha_{\text{bar}}_t}$ ). This is why the distr

import numpy as np
import matplotlib.pyplot as plt
# Robustness: define schedule variables if they are not already defi
# This cell is self-contained: it will create T, betas, alphas, alph

import math
import torch

def _make_linear_beta_schedule(T, beta_start=1e-4, beta_end=0.02, de
    if device is None:
        device = torch.device("cpu")
    betas = torch.linspace(beta_start, beta_end, T, device=device)
    alphas = 1.0 - betas
    alpha_bars = torch.cumprod(alphas, dim=0)
    return betas, alphas, alpha_bars

# Prefer existing globals if present; otherwise create them.
if "device" not in globals():
    device = torch.device("cuda" if torch.cuda.is_available() else "

if "alpha_bars" not in globals() or "T" not in globals():
    T = 200
    betas, alphas, alpha_bars = _make_linear_beta_schedule(T, device
else:
    T = int(len(alpha_bars))

def sample_1d_mixture_np(n=20000, sigma_data=0.35):
    comp = np.random.randint(0, 2, size=n)
    means = np.where(comp==0, -2.0, 2.0)
    x0 = means + sigma_data*np.random.randn(n)
    return x0

# Estimate  $q_{\text{data}}$  by histogram density
x0_s = sample_1d_mixture_np(n=50000, sigma_data=0.35)
```

```

x0_s = sample_1d_mixture_np(n=60000, sigma_data=0.35)

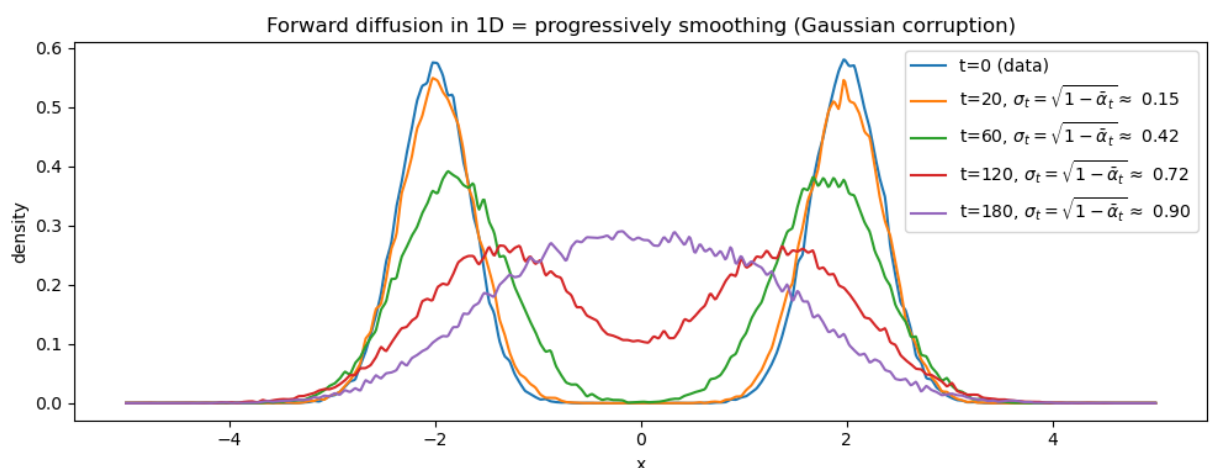
grid = np.linspace(-5, 5, 800)
hist, edges = np.histogram(x0_s, bins=200, range=(-5,5), density=True)
centers = 0.5*(edges[:-1] + edges[1:])
q0 = np.interp(grid, centers, hist)

# Pick a few timesteps and show q(x_t) by Monte Carlo (equivalent to
Ts = [0, int(T*0.1), int(T*0.3), int(T*0.6), int(T*0.9)]
plt.figure(figsize=(10,4))

for t in Ts:
    if t == 0:
        qt = q0
        label = "t=0 (data)"
    else:
        a_bar = float(alphaBars[t].detach().cpu().item())
        n_mc = 60000
        x0_mc = sample_1d_mixture_np(n=n_mc, sigma_data=0.35)
        xt_mc = np.sqrt(a_bar)*x0_mc + np.sqrt(1-a_bar)*np.random.randn(n_mc)
        ht, _ = np.histogram(xt_mc, bins=200, range=(-5,5), density=True)
        qt = np.interp(grid, centers, ht)
        label = f"t={t},  $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$ "
    plt.plot(grid, qt, label=label)

plt.title("Forward diffusion in 1D = progressively smoothing (Gaussian corruption)")
plt.xlabel("x")
plt.ylabel("density")
plt.legend()
plt.tight_layout()
plt.show()

```



✓ Mutual information decay: why x_t “forgets” x_0

A precise way to say “the forward process destroys information” is to look at the **mutual information**:

\$\$

$I(X_0; X_t)$.

\$\$

- If $I(X_0; X_t)$ is large, then knowing x_t tells you a lot about x_0 .
- If $I(X_0; X_t)$ is near 0, then x_t carries almost no information about x_0 .

In forward diffusion, x_t becomes increasingly noisy, so $I(X_0; X_t)$ should decrease with t .

Below we estimate $I(X_0; X_t)$ empirically in 1D using a simple histogram-based estimator (on samples drawn from the simple mixture distribution example).

```
# Empirical mutual information I(X0; Xt) using histogram binning in
# This is a simple estimator for intuition (not a state-of-the-art

import numpy as np
import matplotlib.pyplot as plt
# Robustness: define schedule variables if missing (T, alphaBars)

import math
import torch

def _make_linear_beta_schedule(T, beta_start=1e-4, beta_end=0.02, device=None):
    if device is None:
        device = torch.device("cpu")
    betas = torch.linspace(beta_start, beta_end, T, device=device)
    alphas = 1.0 - betas
    alphaBars = torch.cumprod(alphas, dim=0)
    return betas, alphas, alphaBars

if "device" not in globals():
```

```

    device = torch.device("cuda" if torch.cuda.is_available() else

if "alpha_bars" not in globals() or "T" not in globals():
    T = 200
    betas, alphas, alpha_bars = _make_linear_beta_schedule(T, devic
else:
    T = int(len(alpha_bars))

import numpy as np

def sample_1d_mixture_np(n=20000, sigma_data=0.35):
    comp = np.random.randint(0, 2, size=n)
    means = np.where(comp==0, -2.0, 2.0)
    x0 = means + sigma_data*np.random.randn(n)
    return x0

# estimating mutual information I(X0; Xt) using histogram binning i
def estimate_mi_hist(x, y, bins=80, range_xy=(-5,5)):
    Hxy, xedges, yedges = np.histogram2d(x, y, bins=bins, range=[ra
    px = Hxy.sum(axis=1)
    py = Hxy.sum(axis=0)
    dx = (xedges[1]-xedges[0])
    dy = (yedges[1]-yedges[0])
    pxy = Hxy * dx * dy
    px = px * dx
    py = py * dy
    eps = 1e-12
    pxy = np.maximum(pxy, eps)
    px = np.maximum(px, eps)
    py = np.maximum(py, eps)
    mi = np.sum(pxy * (np.log(pxy) - np.log(px[:,None]) - np.log(py
    return float(mi)

n = 60000
x0 = sample_1d_mixture_np(n=n, sigma_data=0.35)

ts = np.linspace(0, T-1, 20).astype(int)
mi_vals = []

for t in ts:
    if t == 0:
        xt = x0.copy()
    else:
        a_bar = float(alpha_bars[t].detach().cpu().item())
        xt = np.sqrt(a_bar)*x0 + np.sqrt(1-a_bar)*np.random.randn(n
    mi_vals.append(estimate_mi_hist(x0, xt, bins=80, range_xy=(-5,5

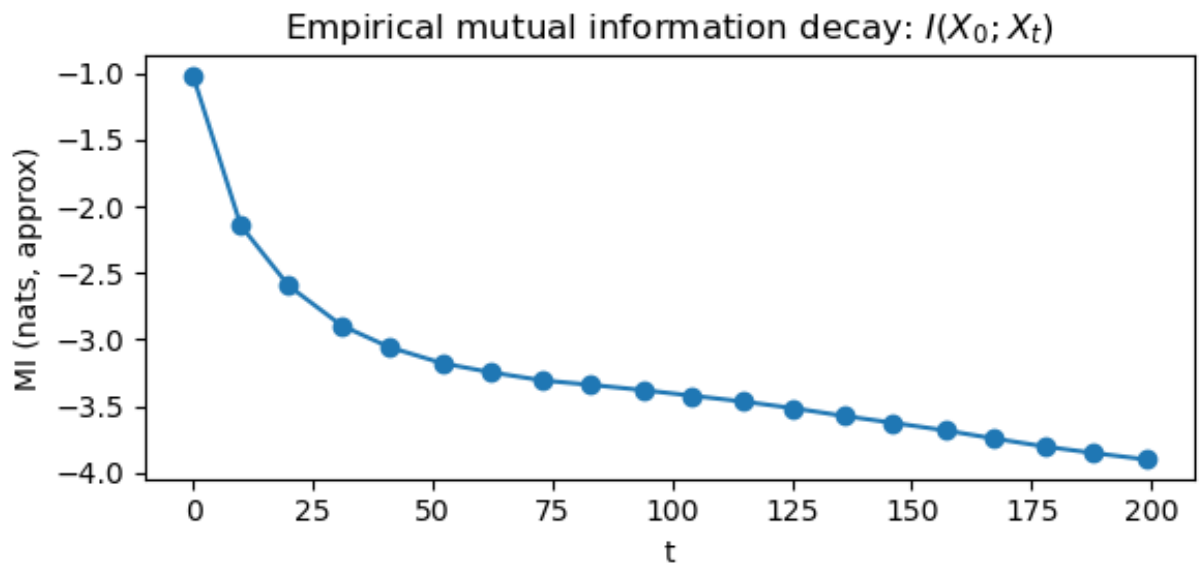
plt.figure(figsize=(6,3))

```



```
plt.plot(ts, mi_vals, marker="o")
plt.title("Empirical mutual information decay:  $I(X_0; X_t)$ ")
plt.xlabel("t")
plt.ylabel("MI (nats, approx)")
plt.tight_layout()
plt.show()

print("MI at t=0:", mi_vals[0])
print("MI at largest t:", mi_vals[-1])
```



MI at t=0: -1.0132524087954993
MI at largest t: -3.9015735459670084

✓ Time scheduling in diffusion: Comparing linear vs cosine

The **noise schedule** (often specified via β_t) controls how quickly the forward process destroys information.

A convenient way to compare schedules is through:

- The cumulative product

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s, \quad \alpha_t = 1 - \beta_t$$

- The **signal-to-noise ratio (SNR)** at time t :

$$\text{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}.$$

Intuition:

- Large SNR: x_t is close to x_0 (easy denoising).
- Small SNR: x_t is close to pure noise (hard denoising).

Below we compare **linear** vs **cosine** schedules by plotting:

1. $\bar{\alpha}_t$ vs t
2. SNR_t vs t (log-scale helps)
3. An empirical estimate of mutual information $I(X_0; X_t)$ vs t (1D simple mixture example)

Note: MI is estimated with a simple histogram estimator for intuition (not a SOTA MI estimator).

```
import numpy as np
import matplotlib.pyplot as plt
import math
import torch

# -----
# 1) Schedules: linear vs cosine
# -----

def make_linear_schedule(T, beta_start=1e-4, beta_end=0.02, device=
    if device is None:
        device = torch.device("cpu")
    betas = torch.linspace(beta_start, beta_end, T, device=device)
```

```

    alphas = 1.0 - betas
    alphaBars = torch.cumprod(alphas, dim=0)
    return betas, alphas, alphaBars

def make_cosine_schedule(T, s=0.008, device=None):
    # Cosine schedule via alpha_bar(t) (Nichol & Dhariwal, 2021):
    #   alpha_bar(t) = cos^2( ((t/T + s)/(1+s)) * pi/2 )
    # Then derive betas from consecutive alpha_bar values.
    if device is None:
        device = torch.device("cpu")
    steps = torch.arange(0, T+1, device=device).float()
    f = (steps / T + s) / (1 + s)
    alpha_bar = torch.cos(f * math.pi / 2) ** 2 # length T+1
    alpha_bar = alpha_bar / alpha_bar[0] # normalize so alp
    betas = []
    for t in range(1, T+1):
        beta_t = 1.0 - (alpha_bar[t] / alpha_bar[t-1])
        betas.append(beta_t)
    betas = torch.stack(betas).clamp(0.0, 0.999) # length T
    alphas = 1.0 - betas
    alphaBars = torch.cumprod(alphas, dim=0)
    return betas, alphas, alphaBars

# Choose a common T for comparison
T_sched = 200
device_local = torch.device("cuda" if torch.cuda.is_available() else

betas_lin, alphas_lin, alphaBars_lin = make_linear_schedule(T_sche
betas_cos, alphas_cos, alphaBars_cos = make_cosine_schedule(T_sche

def snr_from_alpha_bar(alphaBars):
    eps = 1e-12
    return alphaBars / (1.0 - alphaBars + eps)

snr_lin = snr_from_alpha_bar(alphaBars_lin).detach().cpu().numpy()
snr_cos = snr_from_alpha_bar(alphaBars_cos).detach().cpu().numpy()

ab_lin = alphaBars_lin.detach().cpu().numpy()
ab_cos = alphaBars_cos.detach().cpu().numpy()

ts = np.arange(T_sched)

# Plot alpha_bar
plt.figure(figsize=(7,3))
plt.plot(ts, ab_lin, label="linear")
plt.plot(ts, ab_cos, label="cosine")
plt.title(r"Cumulative signal scale: $\bar{\alpha}_t$")
plt.xlabel("t")

```

```

plt.ylabel(r"$\bar{\alpha}_t$")
plt.legend()
plt.tight_layout()
plt.show()

# Plot SNR (log scale)
plt.figure(figsize=(7,3))
plt.plot(ts, snr_lin, label="linear")
plt.plot(ts, snr_cos, label="cosine")
plt.yscale("log")
plt.title(r"SNR over time: $\mathrm{SNR}_t=\bar{\alpha}_t/(1-\bar{\alpha}_t)$")
plt.xlabel("t")
plt.ylabel("SNR")
plt.legend()
plt.tight_layout()
plt.show()

# -----
# 2) Empirical MI:  $I(X_0; X_t)$  for each schedule (1D simple mixture e
# -----

def sample_1d_mixture_np(n=60000, sigma_data=0.35):
    comp = np.random.randint(0, 2, size=n)
    means = np.where(comp==0, -2.0, 2.0)
    x0 = means + sigma_data*np.random.randn(n)
    return x0

def estimate_mi_hist(x, y, bins=80, range_xy=(-5,5)):
    Hxy, xedges, yedges = np.histogram2d(x, y, bins=bins, range=[range_xy])
    px = Hxy.sum(axis=1)
    py = Hxy.sum(axis=0)
    dx = (xedges[1]-xedges[0])
    dy = (yedges[1]-yedges[0])
    pxy = Hxy * dx * dy
    px = px * dx
    py = py * dy
    eps = 1e-12
    pxy = np.maximum(pxy, eps)
    px = np.maximum(px, eps)
    py = np.maximum(py, eps)
    mi = np.sum(pxy * (np.log(pxy) - np.log(px[:,None]) - np.log(py[None,:]))
    return float(mi)

def make_xt_from_x0(x0, alpha_bar_t):
    eps = np.random.randn(*x0.shape)
    return np.sqrt(alpha_bar_t)*x0 + np.sqrt(1.0-alpha_bar_t)*eps

t_grid = np.linspace(0, T_sched-1, 20).astype(int)

```

```

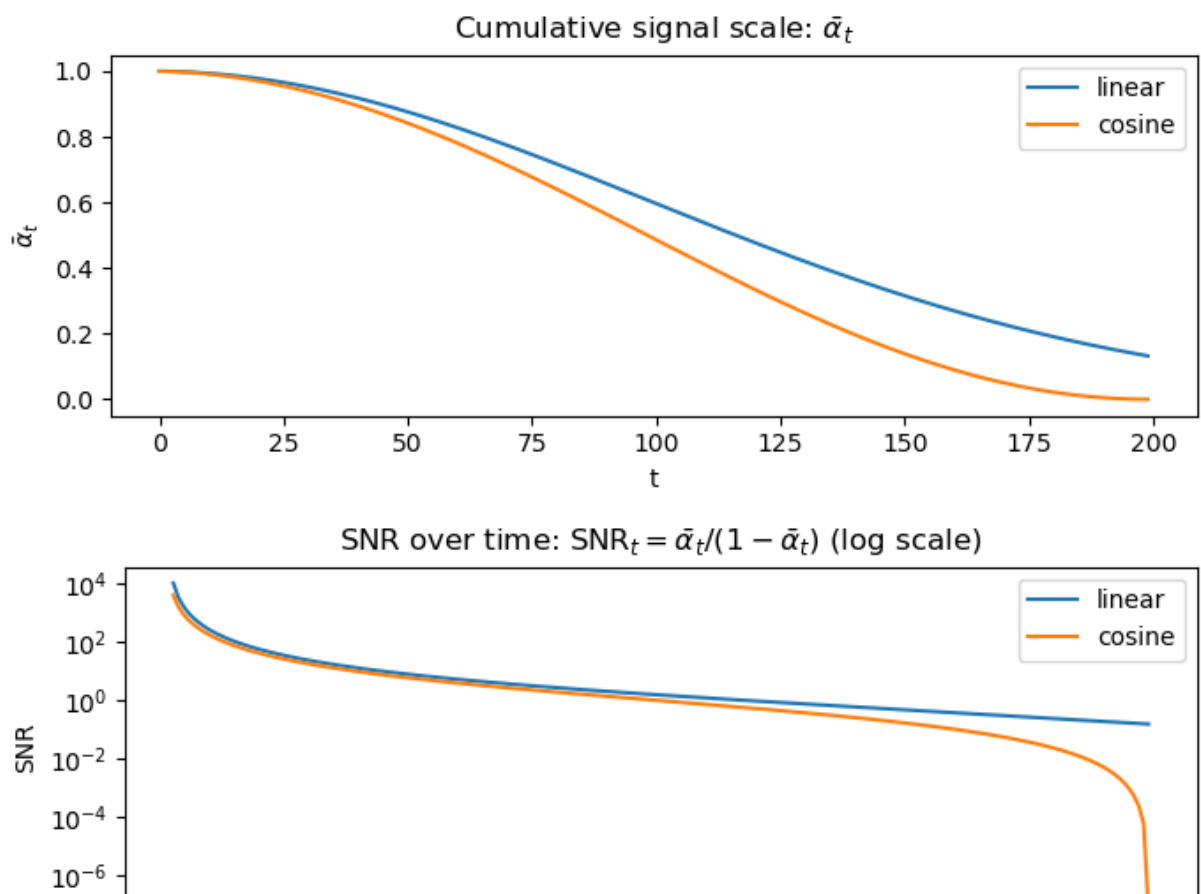
x0 = sample_1d_mixture_np(n=60000, sigma_data=0.35)

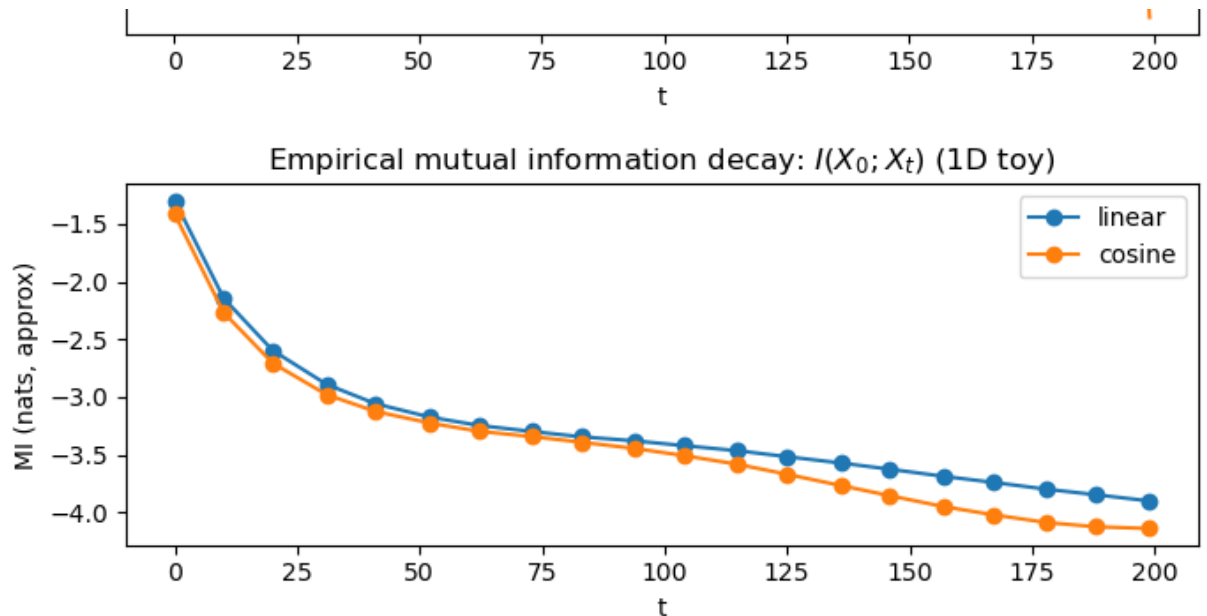
mi_lin = []
mi_cos = []
for t in t_grid:
    abt_lin = float(ab_lin[t])
    abt_cos = float(ab_cos[t])
    xt_lin = make_xt_from_x0(x0, abt_lin)
    xt_cos = make_xt_from_x0(x0, abt_cos)
    mi_lin.append(estimate_mi_hist(x0, xt_lin, bins=80, range_xy=(-
    mi_cos.append(estimate_mi_hist(x0, xt_cos, bins=80, range_xy=(-

plt.figure(figsize=(7,3))
plt.plot(t_grid, mi_lin, marker="o", label="linear")
plt.plot(t_grid, mi_cos, marker="o", label="cosine")
plt.title(r"Empirical mutual information decay:  $I(X_0; X_t)$  (1D t
plt.xlabel("t")
plt.ylabel("MI (nats, approx)")
plt.legend()
plt.tight_layout()
plt.show()

print("MI(t=0) linear vs cosine:", mi_lin[0], mi_cos[0])
print("MI(t=end) linear vs cosine:", mi_lin[-1], mi_cos[-1])

```





MI(t=0) linear vs cosine: -1.2968205636627097 -1.41600123010985
 MI(t=end) linear vs cosine: -3.901975708841169 -4.140799485073025

Reversing the Markov chain in Diffusion

Diffusion can be understood as a two-part story:

We create a forward Markov chain by noising with q

We start from a real data point $x_0 \sim q_{\text{data}}(x)$ and define a *forward* (noising)

Markov chain: $q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1})$.

\$\$

Each transition is simple (Gaussian), e.g. in DDPM:

\$\$

$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t) \mathbf{I})$.

\$\$

As t grows, the distribution becomes close to a standard normal, so we can use:

\$\$

$$p(x_T) = \mathcal{N}(0, I)$$

\$\$

as a convenient “starting point” for generation.

Now we want to invert the noising process

Conceptually, we want to go *backward in time*:

\$\$

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.$$

\$\$

The “true” reverse transition of the forward chain is:

\$\$

$$q(x_{t-1} \mid x_t).$$

\$\$

Why is $q(x_{t-1} \mid x_t)$ intractable?

By Bayes’ rule:

\$\$

$$q(x_{t-1} \mid x_t) = \frac{q(x_t \mid x_{t-1})q(x_{t-1})}{q(x_t)}.$$

\$\$

Even though $q(x_t \mid x_{t-1})$ is Gaussian and known, the **marginals** $q(x_{t-1})$ and $q(x_t)$ are complicated (they depend on the unknown data distribution). So we cannot write or compute $q(x_{t-1} \mid x_t)$ exactly in a way that is usable for sampling.

A common workaround in derivations is to introduce the tractable posterior:

\$\$

$$q(x_{t-1} \mid x_t, x_0),$$

\$\$

which *is* Gaussian in closed form. The posterior requires knowing the original clean point x_0 . However, at generation time, there is no x_0 , so we cannot use this for generation.

We solve this by learning p_θ to approximate the reverse transition

We introduce a model (learned reverse chain):

\$\$

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t),$$

\$\$

where each reverse conditional is parameterized by a neural network (typically Gaussian):

\$\$

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_t).$$

\$\$

Training (via the ELBO / VLB) encourages:

\$\$

$$p_\theta(x_{t-1} \mid x_t) \approx q(x_{t-1} \mid x_t)$$

\$\$

in the sense that each learned reverse step matches the *true* reverse behavior of the forward process.

The reverse process is also a Markov chain

Just like the forward process is Markov (each step depends only on the previous step), the learned reverse process is also Markov:

\$\$

$$p_\theta(x_{t-1} \mid x_t, x_{t+1}, \dots) = p_\theta(x_{t-1} \mid x_t).$$

\$\$

So diffusion sampling is literally “run a Markov chain backward,” but with learned reverse transitions.

Why don’t we end up at the *same* x_0 ?

Because **both** directions are stochastic:

- Forward: we inject fresh Gaussian noise at every step.

- Reverse: we also sample from a distribution at every step.

Even if you start from a noisy point x_t , there are **many** plausible x_{t-1} values that could have produced it. So the reverse step cannot deterministically invert the forward step.

Key idea: diffusion is not trying to invert one specific trajectory to recover the original training example. It is trying to sample a *new* x_0 from a learned distribution $p_\theta(x_0)$ that matches the data distribution.

Why the “true reverse” is intractable

We start from the forward (noising) Markov chain:

\$\$

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1}), \quad q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t) I).$$

\$\$

Marginals of a Markov chain are integrals over the intermediate states

In general, the marginal distribution at time t given x_0 is:

\$\$

$$q(x_t \mid x_0)$$

$$\int q(x_{1:t} \mid x_0) dx_{1:t-1}$$

$$\int \left(\prod_{s=1}^t q(x_s \mid x_{s-1}) \right) dx_{1:t-1}.$$

\$\$

This is an **integral over all intermediate states** $x_1, \dots, x_{t-1}$.

In DDPM, we are lucky: because every transition is Gaussian and linear, this integral simplifies to a closed-form Gaussian

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I).$$

But this tractability is special to the DDPM choice.

The “true reverse” transition $q(x_{t-1} \mid x_t)$ requires the intractable marginals $q(x_t)$ and $q(x_{t-1})$

The time-reversal of the forward chain is defined by the conditional:

$$q(x_{t-1} \mid x_t) = \frac{q(x_t \mid x_{t-1}) q(x_{t-1})}{q(x_t)}.$$

The problem is that the *unconditional* marginals depend on the unknown data distribution $q_{\text{data}}(x_0)$:

\$\$

$q(x_t)$

$\int q(x_t \mid x_0), q_{\text{data}}(x_0), dx_0,$
 $\propto q(x_{t-1})$

$\int q(x_{t-1} \mid x_0), q_{\text{data}}(x_0), dx_0.$

\$\$

Even if $q(x_t \mid x_0)$ is Gaussian, integrating over the full (unknown, complex) data distribution makes $q(x_t)$ a huge, complicated mixture — we cannot evaluate it in closed form.

A compact “one-line” expression showing the intractability is:

\$\$

$q(x_{t-1} \mid x_t)$

$\frac{q(x_t \mid x_{t-1}) \int q(x_{t-1} \mid x_0), q_{\text{data}}(x_0), dx_0}{\int q(x_t \mid x_0), q_{\text{data}}(x_0), dx_0}.$

\$\$

Why we can set $p(x_T) = \mathcal{N}(0, I)$

As t grows, the forward distribution becomes close to standard normal:

\$\$

$q(x_T) \approx \mathcal{N}(0, I).$

\$\$

So we choose a simple prior for the reverse process:

\$\$

$$p(x_T) = \mathcal{N}(0, I), \quad \text{$$}$$

and we learn the reverse conditionals instead of trying to compute $q(x_{t-1} \mid x_t)$ exactly.

How p_θ uses a neural network

Since reversed q is intractable, diffusion defines a **learned reverse Markov chain**:

\$\$

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t), \quad \text{\\quad} \\ p(x_T) = \mathcal{N}(0, I).$$

\$\$

What kind of distribution is $p_\theta(x_{t-1} \mid x_t)$?

In standard DDPM, each reverse step is modeled as a Gaussian:

\$\$

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_t).$$

\$\$

- The **variance** Σ_t is often fixed (or sometimes learned).
- The **mean** $\mu_\theta(x_t, t)$ is predicted by a **neural network**.

So the neural network does **not** output a probability distribution directly. Instead, it outputs the **parameters** of a simple distribution (Gaussian) or more likely the **error**.

Why use a neural network?

The true reverse conditional depends on the unknown data density. A neural network is a flexible function approximator that can learn this dependency from data.

Concretely, the network takes:

- the current noisy sample x_t
- the timestep t

and outputs something like:

- $\epsilon_\theta(x_t, t)$ (predicted noise) -- most common
- $\mu_\theta(x_t, t)$ (predicted mean), or
- $x_{0,\theta}(x_t, t)$ (predicted clean sample)

All parameterizations are algebraically linked.

The common “noise prediction” parameterization

Most implementations train a network to predict the added noise:

\$\$

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

\$\$

The network learns:

\$\$

$$\epsilon_\theta(x_t, t) \approx \epsilon.$$

\$\$

From ϵ_θ , we can compute an estimate of x_0 :

\$\$

$$\hat{x}_0(x_t, t) = \frac{1}{\sqrt{\alpha_t}} (x_t - \sqrt{1 - \alpha_t} \epsilon_\theta(x_t, t)),$$

\$\$

and then plug $\hat{x}_0$ into the closed-form posterior mean to obtain $\mu_\theta(x_t, t)$.

How the neural network is trained in DDPM

This section makes the DDPM training loop explicit: **what are the inputs/targets, what network do we use, and what loss do we optimize?**

What data do we train on?

We start with a dataset of clean samples: \$\$

$x_0 \sim q_{\text{data}}(x)$.

\$\$

DDPM training does **not** require real “noisy trajectories” stored in a dataset. Instead, we **synthesize** training pairs on the fly using the closed-form forward marginal:

1. Sample a clean datapoint x_0 from the dataset.
2. Sample a timestep $t \sim \text{Uniform}\{1, \dots, T\}$.
3. Sample Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$.
4. Construct a noisy version:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

So each training example is effectively a triple:

\$\$

(x_t, t, ϵ)

\$\$

derived from a clean x_0 .

Key point: the dataset only provides x_0 . The noisy x_t and the target noise ϵ are generated synthetically.

What does the neural network predict?

The most common DDPM parameterization is **noise prediction**:

\$\$

$\epsilon_{\theta}(x_t, t) \approx \epsilon$.

\$\$

Why this is convenient:

- The target ϵ is always standard normal (nice scale).
- The loss becomes a simple mean-squared error.
- From ϵ_{θ} , we can reconstruct $\hat{x}_0$ and the reverse mean $\mu_{\theta}(x_t, t)$.

The DDPM training objective

The standard “simple loss” is:

\$\$

$\mathcal{L}_{\text{simple}}(\theta)$

$$\mathbb{E}_{x_0 \sim q(\text{data}), t, \epsilon \sim \mathcal{N}(0, I)} \left[\left| \epsilon - \epsilon_\theta(x_t, t) \right|^2 \right]$$

\$\$

This is equivalent (up to weighting choices) to optimizing the variational bound terms for Gaussian transitions.

What is the architecture?

In full image DDPMs

For images, the standard architecture is a **U-Net** conditioned on the timestep:

- Input: image-like x_t (shape $H \times W \times C$) + timestep t
- Output: predicted noise image $\epsilon_\theta(x_t, t)$ (same shape as x_t)
- Uses **downsampling/upsampling** with skip connections to preserve spatial detail
- Includes **time embedding** (sinusoidal or learned) injected into residual blocks
- Often includes attention blocks at certain resolutions

Training loop (high level)

Repeat:

1. sample batch of x_0
2. sample t
3. sample ϵ
4. form x_t
5. predict $\hat{\epsilon} = \epsilon_\theta(x_t, t)$
6. minimize MSE: $\|\epsilon - \hat{\epsilon}\|^2$

At generation time, we do not know the true ϵ that produced x_t . So we train a neural net $\epsilon_\theta(x_t, t)$ to predict it.

Define the model mean:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right). \quad (9)$$

Then we define the learned reverse transition:

$$p_{\theta}(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \sigma_t^2),$$

with σ_t^2 chosen (often fixed) to match the forward/posterior variance behavior.

Summary

- The forward marginal gives $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$.
- Solving for x_0 shows knowing ϵ is equivalent to knowing the clean signal.
- The Gaussian posterior mean $\tilde{\mu}(x_t, x_0)$ can be rewritten as a function of ϵ . (out of scope for this course)
- Therefore, predicting ϵ with a neural network directly yields the reverse mean μ_{θ} .

✓ Representing Time: Sinusoidal time encoding

Diffusion models must be conditioned on the timestep t because denoising at:

- small t (slightly noisy) and
- large t (almost pure noise)

are fundamentally different tasks.

Rather than feeding the integer t directly, we map it to a **vector embedding** $\text{emb}(t) \in \mathbb{R}^d$.

The standard sinusoidal embedding

For $k = 0, \dots, \frac{d}{2} - 1$:

\$\$

$$\text{emb}(t)_{2k} = \sin(t \cdot \omega_k), \quad \text{emb}(t)_{2k+1} = \cos(t \cdot \omega_k),$$

\$\$

with geometrically spaced frequencies:

\$\$

$$\omega_k = 10000^{-k/(d/2)}.$$

\$\$

Summary

- Low-frequency dimensions vary slowly with t (capture coarse time).
- High-frequency dimensions vary quickly with t (capture fine time).
- Nearby timesteps get nearby embeddings, but the embedding remains expressive across a wide range.

This is the same idea as Transformer positional encodings, reused here for timesteps.

```
import numpy as np
import matplotlib.pyplot as plt
import math

def sinusoidal_time_embedding(t, dim=64, max_period=10000.0):
```



```

# t: array-like of shape (N,) timesteps
# returns: (N, dim) embedding
t = np.asarray(t).astype(np.float32)
half = dim // 2
# frequencies
freqs = np.exp(-math.log(max_period) * np.arange(0, half, dtype=
args = t[:, None] * freqs[None, :] # (N, half)
emb = np.concatenate([np.sin(args), np.cos(args)], axis=1) # (
if dim % 2 == 1:
    emb = np.concatenate([emb, np.zeros((len(t),1), dtype=np.fl
return emb

# Example 1: visualize embeddings across timesteps as an "image"
T_vis = 200
ts = np.arange(T_vis)
dim = 64
E = sinusoidal_time_embedding(ts, dim=dim)

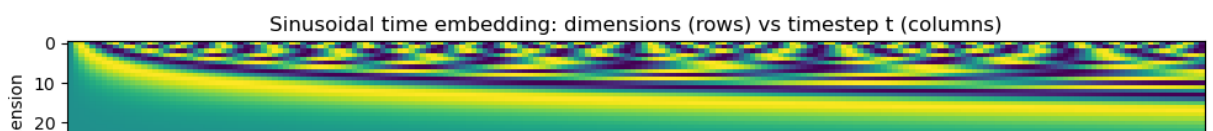
plt.figure(figsize=(10,3))
plt.imshow(E.T, aspect="auto")
plt.title("Sinusoidal time embedding: dimensions (rows) vs timestep
plt.xlabel("t")
plt.ylabel("embedding dimension")
plt.tight_layout()
plt.show()

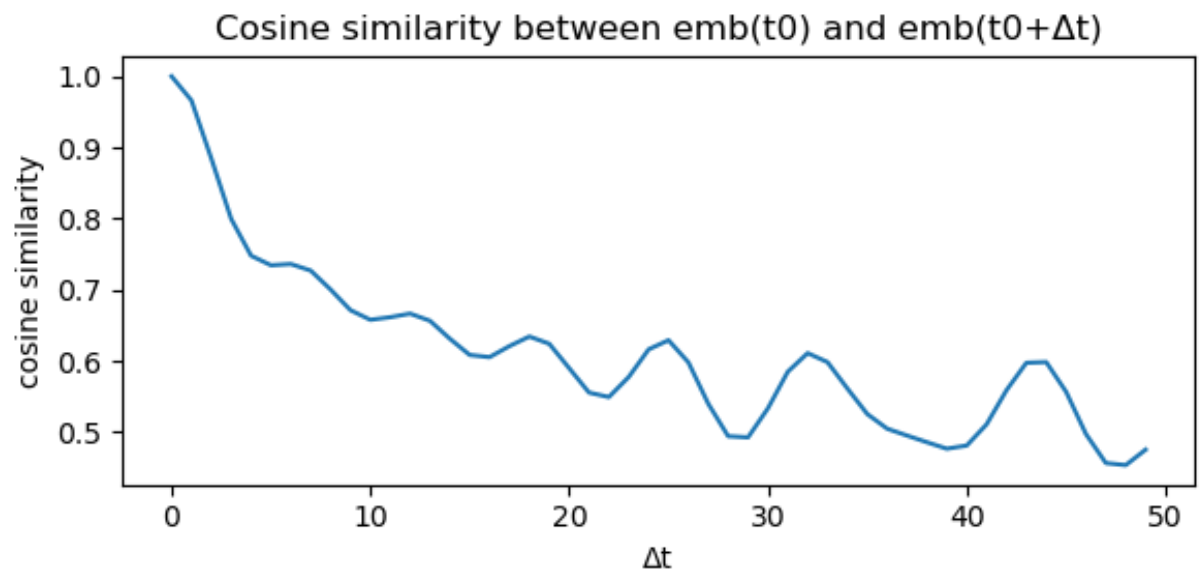
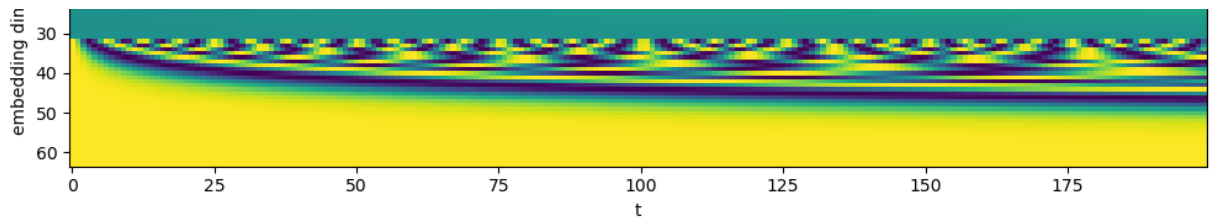
# Example 2: show that nearby timesteps are close (cosine similarit
def cosine_sim(a, b, eps=1e-12):
    a = a / (np.linalg.norm(a) + eps)
    b = b / (np.linalg.norm(b) + eps)
    return float(np.dot(a, b))

t0 = 50
deltas = np.arange(0, 50)
sims = [cosine_sim(E[t0], E[t0+dt]) for dt in deltas]

plt.figure(figsize=(6,3))
plt.plot(deltas, sims)
plt.title("Cosine similarity between emb(t0) and emb(t0+Δt)")
plt.xlabel("Δt")
plt.ylabel("cosine similarity")
plt.tight_layout()
plt.show()

```





✓ SiLU (Swish) Activation

SiLU stands for **Sigmoid Linear Unit**. It is also known as **Swish** (when written as $x \cdot \sigma(x)$). It is a smooth, non-monotonic activation that often works well in modern deep nets (including diffusion U-Nets).

Definition

Let the sigmoid be:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Then the SiLU activation is:

$$\text{SiLU}(x) = x \sigma(x) = \frac{x}{1 + e^{-x}}.$$

Key intuition

- For large positive x , $\sigma(x) \approx 1$, so:

$$\text{SiLU}(x) \approx x$$

- For large negative x , $\sigma(x) \approx 0$, so:

$$\text{SiLU}(x) \approx 0$$

- Near $x = 0$, it behaves like a soft, smooth gating of x .

Derivative (useful for backprop)

Using the product rule:

$$\frac{d}{dx} \text{SiLU}(x) = \sigma(x) + x \sigma(x) (1 - \sigma(x)).$$

This derivative is smooth everywhere, which can help optimization compared to activations with kinks.

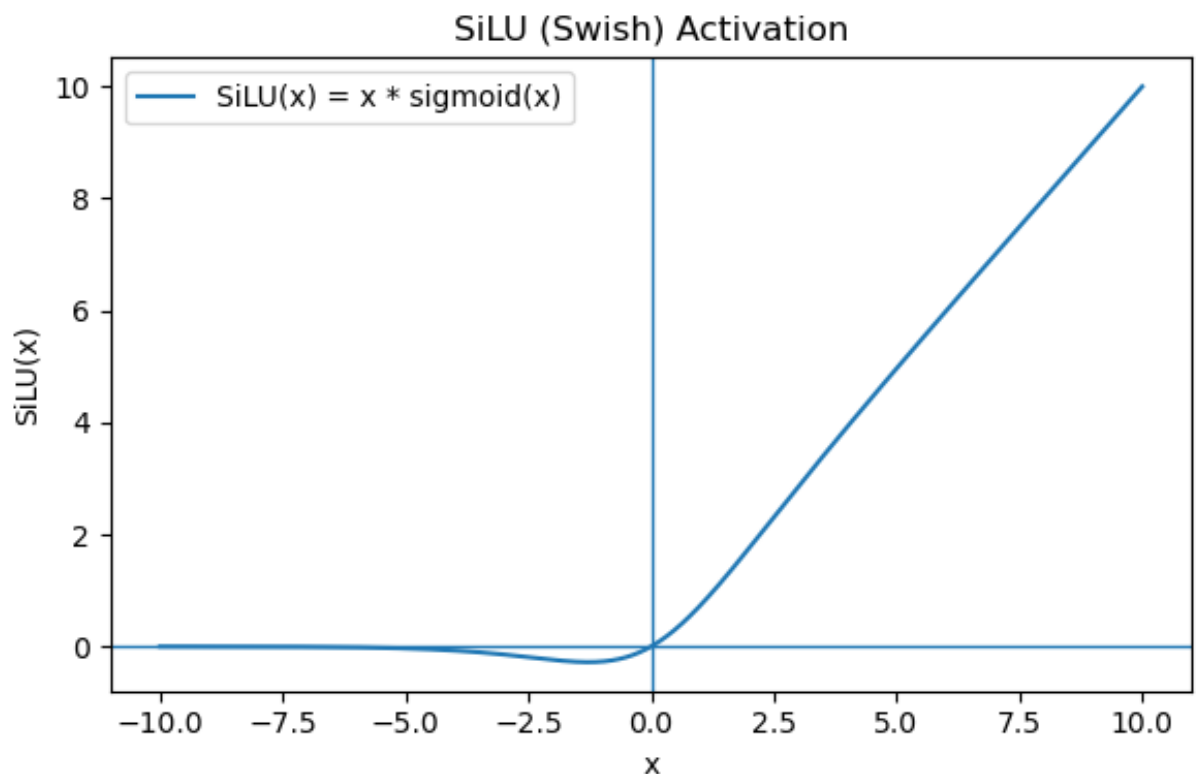
```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def silu(x):
    return x * sigmoid(x)

x = np.linspace(-10, 10, 1000)
y = silu(x)

plt.figure(figsize=(6, 4))
plt.plot(x, y, label="SiLU(x) = x * sigmoid(x)")
plt.axhline(0, linewidth=1)
plt.axvline(0, linewidth=1)
plt.title("SiLU (Swish) Activation")
plt.xlabel("x")
plt.ylabel("SiLU(x)")
plt.legend()
plt.tight_layout()
plt.show()
```



✓ 1D DDPM Example Implementation

We intentionally use **scalar** data $x \in \mathbb{R}$ (shape $(B, 1)$):

- Data distribution $q_{\text{data}}(x_0)$: a simple **two-bump** mixture.
- Forward diffusion: add Gaussian noise with a schedule $\{\beta_t\}$.
- Model: a tiny MLP that predicts noise $\epsilon_\theta(x_t, t)$.
- Reverse diffusion (DDPM sampling): start from $x_T \sim \mathcal{N}(0, 1)$ and denoise back to x_0 .
- Denoising trajectory: show how a single scalar evolves over timesteps.

```
# --- imports (self-contained) ---
import math
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(0)
np.random.seed(0)

# data: scalar mixture of two Gaussians ("two bumps")
def sample_two_bumps(n, sigma_data=0.35, device=device):
    comp = torch.randint(0, 2, (n,), device=device)
    means = torch.where(comp == 0, torch.tensor(-2.0, device=device),
                        torch.tensor(2.0, device=device))
    x0 = means + sigma_data * torch.randn(n, device=device)
    return x0.view(-1, 1)  # (B,1)

# schedule: linear betas
# Here we use a linear beta schedule for simplicity, but many other

class DiffusionSchedule:
    def __init__(self, T=200, beta_start=1e-4, beta_end=0.02, device=device):
        self.T = int(T)
        self.device = device
        self.betas = torch.linspace(beta_start, beta_end, self.T, device=device)
        self.alphas = 1.0 - self.betas
        self.alpha_bars = torch.cumprod(self.alphas, dim=0)

    def extract(self, a, t, x_shape):
        # a: (T,), t: (B,) -> broadcastable to x_shape
```

```

        out = a.gather(0, t) # (B,)
        return out.view(-1, *([1] * (len(x_shape) - 1)))

# Create a schedule for 200 steps
sched = DiffusionSchedule(T=200, device=device)

# Forward marginal  $q(x_t | x_0)$  in closed form
def q_sample(x0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x0)
    a_bar = sched.extract(sched.alpha_bars, t, x0.shape)
    xt = torch.sqrt(a_bar) * x0 + torch.sqrt(1.0 - a_bar) * noise
    return xt, noise

# Time embedding + epsilon model (tiny MLP)
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim=64):
        super().__init__()
        self.dim = dim
    def forward(self, t):
        half = self.dim // 2
        freqs = torch.exp(-math.log(10000) * torch.arange(0, half,
            args = t.float().unsqueeze(1) * freqs.unsqueeze(0)
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=1)
        if self.dim % 2 == 1:
            emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], di
        return emb

# SiLU activations are smooth and work well for diffusion models.
# The model takes in the noisy input  $x_t$  and the time embedding, an

class EpsMLP(nn.Module):
    def __init__(self, tdim=64, hidden=128):
        super().__init__()
        self.tenc = SinusoidalTimeEmbedding(tdim)
        self.net = nn.Sequential(
            nn.Linear(1 + tdim, hidden),
            nn.SiLU(),
            nn.Linear(hidden, hidden),
            nn.SiLU(),
            nn.Linear(hidden, 1),
        )
    # The model concatenates the noisy input  $x_t$  with the time embe
    # dim=1 for scalar data, tdim for time embedding, hidden for ML

    def forward(self, x, t):
        temb = self.tenc(t)
        return self.net(torch.cat([x, temb], dim=1))

```

```

# Create the epsilon model and optimizer
eps_model = EpsMLP(tdim=64, hidden=128).to(device)
opt = torch.optim.AdamW(eps_model.parameters(), lr=2e-4)

# Training: predict noise (DDPM objective)
def train_step(batch=512):
    # train the model for one step on a batch of data sampled from
    eps_model.train()

    # grab an x0 batch, sample random timesteps, get noisy xt and n
    x0 = sample_two_bumps(batch)
    t = torch.randint(0, sched.T, (batch,)), device=device, dtype=tc

    # q sample return the xt and the corresponding noise eps used t
    # The model tries to predict this eps given xt and t.

    xt, eps = q_sample(x0, t)
    eps_pred = eps_model(xt, t)

    # loss represents how well the model's predicted noise matches
    loss = F.mse_loss(eps_pred, eps)
    opt.zero_grad()
    loss.backward()
    opt.step()
    return float(loss.item())

@torch.no_grad()
def eval_batch(batch=512):
    eps_model.eval()
    x0 = sample_two_bumps(batch)
    t = torch.randint(0, sched.T, (batch,)), device=device, dtype=tc
    xt, eps = q_sample(x0, t)
    eps_pred = eps_model(xt, t)
    loss = F.mse_loss(eps_pred, eps)
    return float(loss.item()), x0, xt, eps, eps_pred, t

losses = []
for it in range(2000):
    losses.append(train_step(512))
    if (it+1) % 500 == 0:
        print(f"iter {it+1}/2000 | loss {np.mean(losses[-200:]):.4f}")

plt.figure(figsize=(6,3))
plt.plot(losses)
plt.title("Scalar 1D diffusion: training loss (predicting noise)")
plt.xlabel("iteration")
plt.ylabel("MSE")

```

```

plt.tight_layout()
plt.show()

# DDPM reverse step + sampling
@torch.no_grad()
def ddpm_step(xt, t):
    beta_t = sched.extract(sched.betas, t, xt.shape)
    alpha_t = sched.extract(sched.alphas, t, xt.shape)
    a_bar_t = sched.extract(sched.alpha_bars, t, xt.shape)

    # get the predicted noise from the model, and compute the mean
    eps = eps_model(xt, t)
    mean = (1.0 / torch.sqrt(alpha_t)) * (xt - (beta_t / torch.sqrt(alpha_t)) * eps)

    if (t == 0).all():
        return mean
    z = torch.randn_like(xt)

    # return a sample from the reverse distribution by adding noise
    return mean + torch.sqrt(beta_t) * z

# To sample from the model, we start with pure noise  $x_T \sim N(0,1)$  and
# This simulates the reverse diffusion process and should yield samples
# the model has learned well.

@torch.no_grad()
def sample_ddpm(n=60000):
    xt = torch.randn(n, 1, device=device) #  $x_T \sim N(0,1)$ 
    for time in range(sched.T-1, -1, -1):
        t = torch.full((n,), time, device=device, dtype=torch.long)
        xt = ddpm_step(xt, t)
    return xt

# Compare true data vs generated
with torch.no_grad():
    x_true = sample_two_bumps(60000).squeeze(1).cpu().numpy()
    x_gen = sample_ddpm(60000).squeeze(1).cpu().numpy()

bins = np.linspace(-5, 5, 200)
plt.figure(figsize=(10,3))
plt.hist(x_true, bins=bins, density=True, alpha=0.5, label="true data")
plt.hist(x_gen, bins=bins, density=True, alpha=0.5, label="DDPM samples")
plt.axvline(0, linestyle="--")
plt.title("Scalar 1D DDPM: reverse sampling recovers the two bumps")
plt.xlabel("x")
plt.ylabel("density")
plt.legend()
plt.tight_layout()

```



```
plt.show()

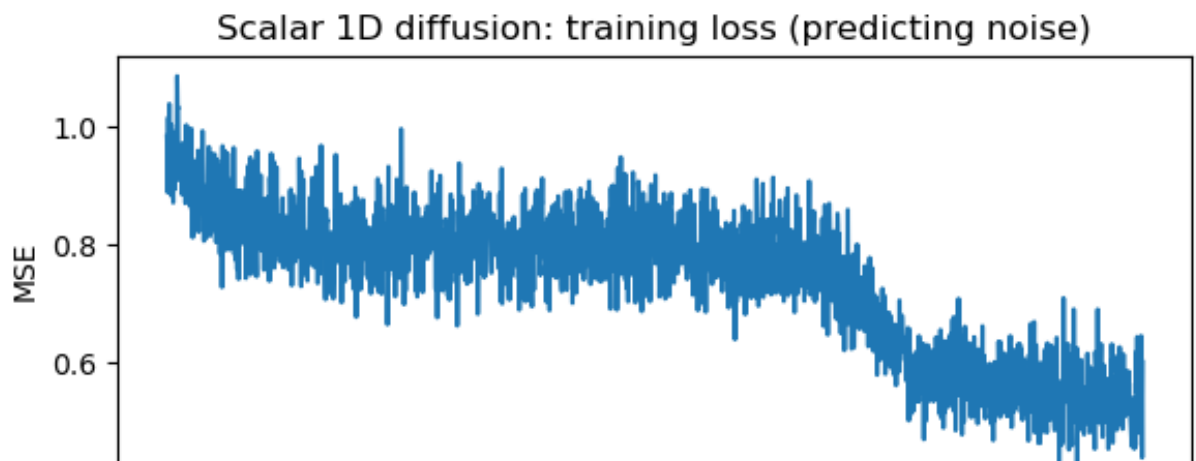
# Showing Denoising trajectory for ONE scalar ---
@torch.no_grad()
def one_scalar_trajectory(snap_ts=None):
    if snap_ts is None:
        snap_ts = [sched.T-1, int(0.7*(sched.T-1)), int(0.4*(sched.
xt = torch.randn(1, 1, device=device)
snaps = {}
snap_set = set(snap_ts)
for time in range(sched.T-1, -1, -1):
    t = torch.full((1,), time, device=device, dtype=torch.long)
    xt = ddpm_step(xt, t)
    if time in snap_set:
        snaps[time] = float(xt.item())
return snaps

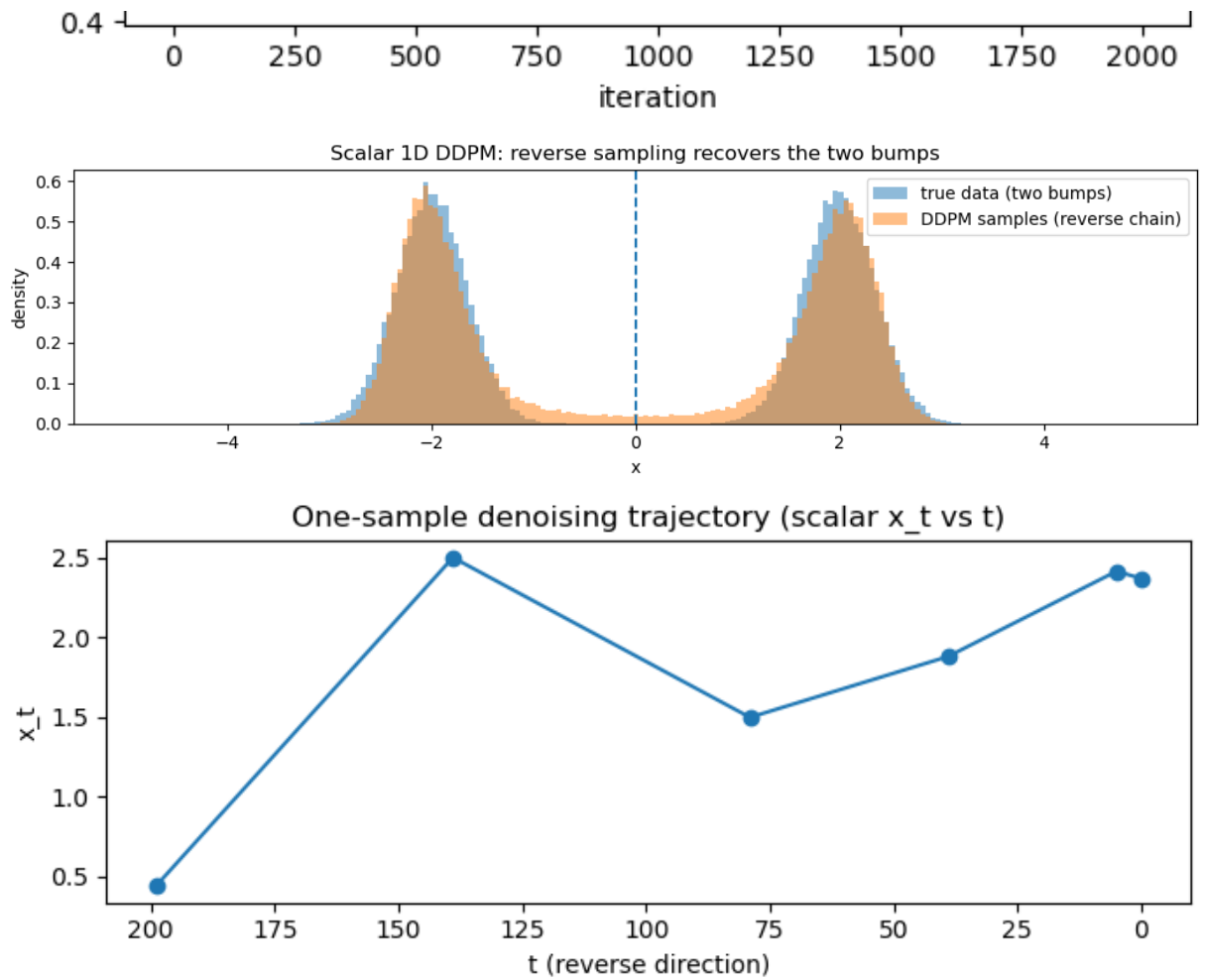
snaps = one_scalar_trajectory()
ts = sorted(snaps.keys(), reverse=True)
vals = [snaps[t] for t in ts]

plt.figure(figsize=(7,3))
plt.plot(ts, vals, marker="o")
plt.gca().invert_xaxis()
plt.title("One-sample denoising trajectory (scalar x_t vs t)")
plt.xlabel("t (reverse direction)")
plt.ylabel("x_t")
plt.tight_layout()
plt.show()

print("Trajectory snapshots:", snaps)
```

```
iter 500/2000 | loss 0.8030
iter 1000/2000 | loss 0.8043
iter 1500/2000 | loss 0.7159
iter 2000/2000 | loss 0.5438
```





Trajectory snapshots: {199: 0.43972718715667725, 139: 2.499115228652

✓ Forward vs reverse histograms (intermediate timesteps)

These plots make the *Markov chain intuition* concrete:

- **Forward (noising)**: start from $x_0 \sim q_{\text{data}}$. As t increases, the bimodal density **smooths** and approaches $\mathcal{N}(0, 1)$.
- **Reverse (denoising)**: start from $x_T \sim \mathcal{N}(0, 1)$. As t decreases, the density becomes **bimodal** again.

We plot intermediate **histograms** $q(x_t)$ (forward) and $p_\theta(x_t)$ (reverse sampling snapshots).

```
import numpy as np
import matplotlib.pyplot as plt
import torch

# Choose a few snapshot timesteps
snap_ts = [0, int(0.1*(sched.T-1)), int(0.3*(sched.T-1)), int(0.6*(
snap_ts = sorted(list(set(snap_ts)))

bins = np.linspace(-5, 5, 200)

# ----- Forward: q(x_t) at several t -----
with torch.no_grad():
    n = 80000
    x0 = sample_two_bumps(n) # (n,1)
    grid_hists = []
    labels = []
    for t_int in snap_ts:
        t = torch.full((n,), t_int, device=device, dtype=torch.long)
        xt, _ = q_sample(x0, t)
        xt_np = xt.squeeze(1).cpu().numpy()
        h, _ = np.histogram(xt_np, bins=bins, density=True)
        grid_hists.append(h)
        if t_int == 0:
            labels.append("t=0 (data)")
        elif t_int == sched.T-1:
            labels.append(f"t={t_int} (~noise)")
        else:
            labels.append(f"t={t_int}")

    bin_centers = 0.5*(bins[:-1] + bins[1:])

plt.figure(figsize=(10,3))
```

```

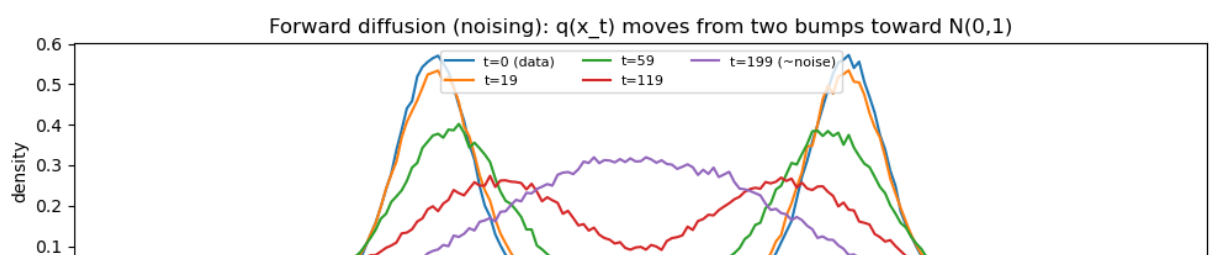
for h, lab in zip(grid_hists, labels):
    plt.plot(bin_centers, h, label=lab)
plt.title("Forward diffusion (noising):  $q(x_t)$  moves from two bumps
plt.xlabel("x")
plt.ylabel("density")
plt.legend(ncol=3, fontsize=8)
plt.tight_layout()
plt.show()

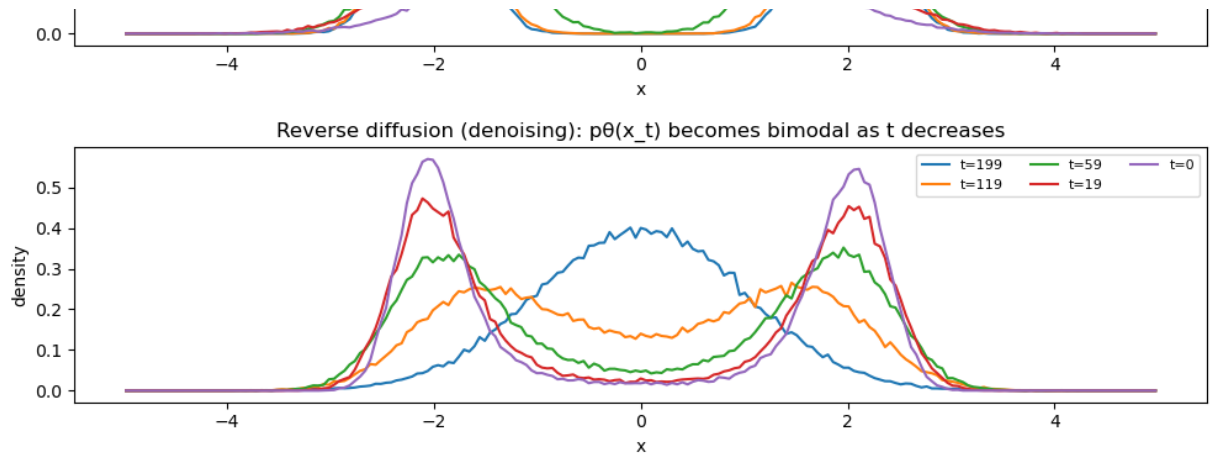
# ----- Reverse:  $p_\theta(x_t)$  snapshots during DDPM sampling -
@torch.no_grad()
def sample_ddpm_with_snaps(n=80000, snap_ts=None):
    if snap_ts is None:
        snap_ts = [sched.T-1, int(0.6*(sched.T-1)), int(0.3*(sched.
    snap_set = set(snap_ts)
    xt = torch.randn(n, 1, device=device) #  $x_T$ 
    snaps = {}
    for time in range(sched.T-1, -1, -1):
        t = torch.full((n,), time, device=device, dtype=torch.long)
        xt = ddpm_step(xt, t)
        if time in snap_set:
            snaps[time] = xt.detach().clone()
    return xt, snaps

rev_ts = [sched.T-1, int(0.6*(sched.T-1)), int(0.3*(sched.T-1)), in
with torch.no_grad():
    x_final, snaps = sample_ddpm_with_snaps(n=80000, snap_ts=rev_ts

rev_ts_sorted = sorted(snaps.keys(), reverse=True)
plt.figure(figsize=(10,3))
for t_int in rev_ts_sorted:
    xt_np = snaps[t_int].squeeze(1).cpu().numpy()
    h, _ = np.histogram(xt_np, bins=bins, density=True)
    plt.plot(bin_centers, h, label=f"t={t_int}")
plt.title("Reverse diffusion (denoising):  $p_\theta(x_t)$  becomes bimodal a
plt.xlabel("x")
plt.ylabel("density")
plt.legend(ncol=3, fontsize=8)
plt.tight_layout()
plt.show()

```





✓ Single-scalar noising and denoising trajectory

We now track **one single scalar sample**:

- **Forward trajectory**: start from one clean x_0 and apply the *closed-form forward marginal* $q(x_t|x_0)$.
- **Reverse trajectory**: start from pure noise $x_T \sim \mathcal{N}(0, 1)$ and apply the DDPM reverse chain.

This makes the stochastic Markov chain behavior visually concrete at the *individual sample* level.

```
import torch
import numpy as np
import matplotlib.pyplot as plt

# ----- Forward trajectory (one scalar) -----
with torch.no_grad():
```

```

x0_single = sample_two_bumps(1) # (1,1)
forward_vals = []
ts_forward = list(range(sched.T))

for t_int in ts_forward:
    t = torch.full((1,), t_int, device=device, dtype=torch.long)
    xt, _ = q_sample(x0_single, t)
    forward_vals.append(float(xt.item()))

# Plot forward trajectory
plt.figure(figsize=(7,3))
plt.plot(ts_forward, forward_vals)
plt.title("Forward noising trajectory for one scalar sample")
plt.xlabel("t")
plt.ylabel("x_t")
plt.tight_layout()
plt.show()

print("Initial clean value x0:", float(x0_single.item()))
print("Final noisy value x_T:", forward_vals[-1])

# ----- Reverse trajectory (one scalar) -----
with torch.no_grad():
    xt = torch.randn(1,1, device=device) # x_T
    reverse_vals = []
    ts_reverse = list(range(sched.T-1, -1, -1))

    for t_int in ts_reverse:
        t = torch.full((1,), t_int, device=device, dtype=torch.long)
        xt = ddpm_step(xt, t)
        reverse_vals.append(float(xt.item()))

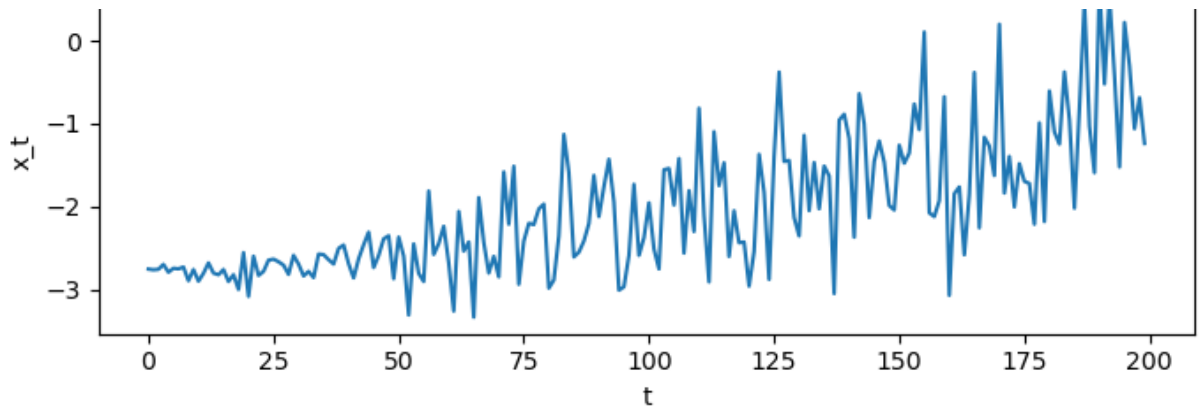
# Plot reverse trajectory
plt.figure(figsize=(7,3))
plt.plot(ts_reverse, reverse_vals)
plt.gca().invert_xaxis()
plt.title("Reverse denoising trajectory for one scalar sample (DDPM)")
plt.xlabel("t (reverse direction)")
plt.ylabel("x_t")
plt.tight_layout()
plt.show()

print("Initial noise x_T:", reverse_vals[0])
print("Final generated x0:", reverse_vals[-1])

```

Forward noising trajectory for one scalar sample

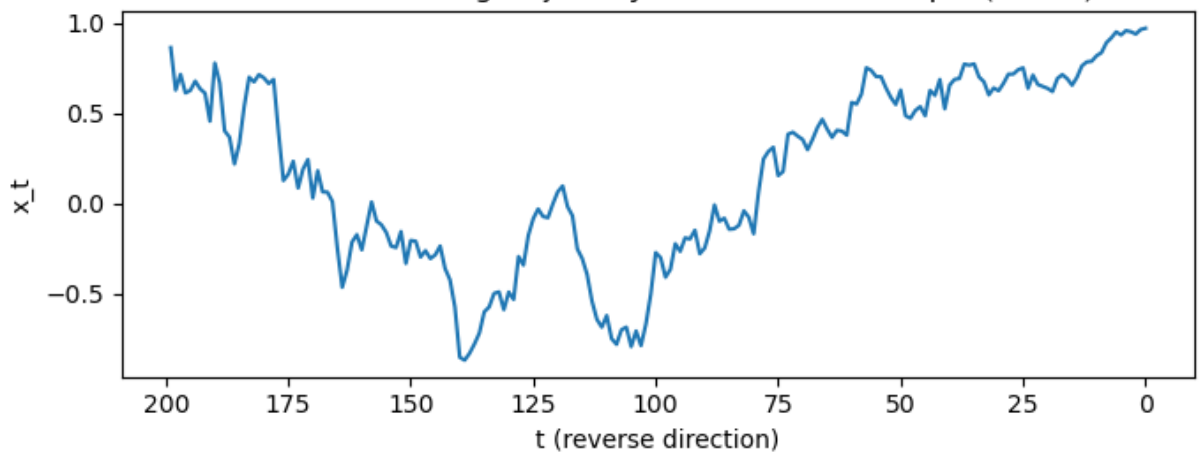




Initial clean value x_0 : -2.758697986602783

Final noisy value x_T : -1.237990379333496

Reverse denoising trajectory for one scalar sample (DDPM)



Initial noise x_T : 0.8657407164573669

Final generated x_0 : 0.9730471968650818

✓ What is a U-Net and why is it used in diffusion?

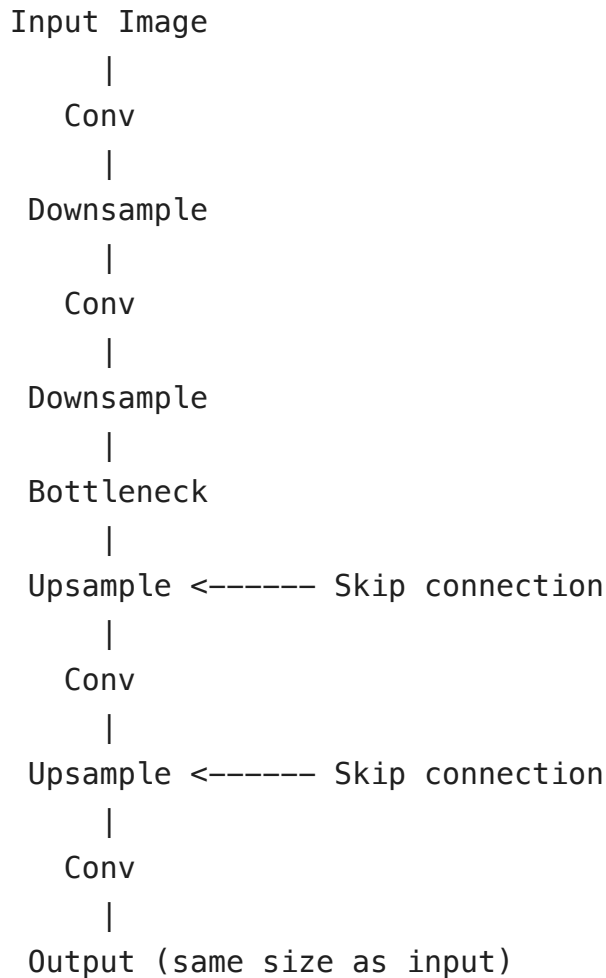
When moving from 1D examples to real images, the neural network used in DDPM is typically a **U-Net**.

What is a U-Net?

A U-Net is a convolutional neural network with:

- A **downsampling path** (encoder)
- An **upsampling path** (decoder)
- **Skip connections** between matching resolutions

Its structure looks like a "U":



Key properties:

- Preserves **spatial resolution**
- Captures both **global context** (deep layers)
- Retains **fine details** (skip connections)

Why is U-Net ideal for diffusion?

In diffusion, the network must predict:

$$\epsilon_{\theta}(x_t, t)$$

where:

- Input x_t is a noisy image
- Output is a noise image of the **same size**

So we need a model that:

- Takes an image as input

- Outputs an image of identical resolution
- Understands multi-scale structure
- Preserves spatial precision

U-Net satisfies all of these requirements.

If we used a simple CNN without skip connections:

- Fine details would be lost
- High-resolution structure would degrade

Diffusion requires **pixel-accurate denoising**, which is why U-Net is the dominant architecture.

Why do we need time embeddings?

The network must know **how much noise is present**.

Predicting noise at:

- $t = 10$ (slightly noisy)
- $t = 900$ (almost pure noise)

are completely different tasks.

Without knowing t , the network cannot distinguish:

- “small correction” regime
- “global reconstruction” regime

So we provide the timestep t as input.

How is time provided to the network?

We convert integer timestep t into a **continuous embedding vector**.

Common choice: sinusoidal embedding (like in Transformers):

\$\$

$$\text{emb}(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \dots]$$

\$\$

This produces a high-dimensional smooth representation of time.

The embedding is then:

- Added to feature maps
- Injected into residual blocks

- Used to modulate activations (FiLM-style)

Why sinusoidal embeddings?

Because they:

- Provide smooth interpolation across time
- Encode relative position information
- Allow generalization across timesteps

This is similar to positional encodings in Transformers.

Summary

In diffusion:

- U-Net handles **spatial structure**
- Time embeddings handle **noise level conditioning**

Together, they let the model learn:

\$\$

$(x_t, t) \mapsto \epsilon, \theta(x_t, t)$ \$\$

This is why diffusion models are sometimes described as:

Time-conditioned denoising networks.

```
# Example of using the sinusoidal time embedding in a PyTorch model
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim: int):
        super().__init__()
        self.dim = dim

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        # t: (B,) integer timesteps
        half = self.dim // 2
        freqs = torch.exp(-math.log(10000) * torch.arange(0, half,
            args = t.float().unsqueeze(1) * freqs.unsqueeze(0) # (B, h
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=1)
        if self.dim % 2 == 1:
```

```

        emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], dim=1)
    return emb  # (B, dim)

class ResBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int, tdim: int):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1)
        self.act = nn.SiLU()
        self.tproj = nn.Linear(tdim, out_ch)  # project time embedding
        self.skip = nn.Identity() if in_ch == out_ch else nn.Conv2d(in_ch, out_ch, kernel_size=1, padding=0)

    def forward(self, x: torch.Tensor, temb: torch.Tensor) -> torch.Tensor:
        # temb: (B, tdim)
        h = self.act(self.conv1(x))
        # inject time embedding as a per-channel bias (FiLM-lite)
        t = self.tproj(temb).unsqueeze(-1).unsqueeze(-1)  # (B, out_ch, 1, 1)
        h = h + t
        h = self.act(self.conv2(h))
        return h + self.skip(x)

class Downsample(nn.Module):
    def __init__(self, ch: int):
        super().__init__()
        self.conv = nn.Conv2d(ch, ch, kernel_size=3, stride=2, padding=1)

    def forward(self, x): return self.conv(x)

class Upsample(nn.Module):
    def __init__(self, ch: int):
        super().__init__()
        self.conv = nn.Conv2d(ch, ch, kernel_size=3, padding=1)

    def forward(self, x):
        x = F.interpolate(x, scale_factor=2, mode="nearest")
        return self.conv(x)

class TinyUNet2D(nn.Module):
    def __init__(self, in_ch=1, base_ch=32, tdim=128, out_ch=None):
        super().__init__()
        if out_ch is None:
            out_ch = in_ch  # predict epsilon with same channels as input

        self.time = SinusoidalTimeEmbedding(tdim)
        self.time_mlp = nn.Sequential(
            nn.Linear(tdim, tdim*4),
            nn.SiLU(),
            nn.Linear(tdim*4, tdim),
        )

```

```

# Encoder
self.in_conv = nn.Conv2d(in_ch, base_ch, kernel_size=3, padding=1)
self.enc1 = ResBlock(base_ch, base_ch, tdim)
self.down = Downsample(base_ch)

# Bottleneck
self.mid = ResBlock(base_ch, base_ch, tdim)

# Decoder
self.up = Upsample(base_ch)
# concat skip => channels double
self.dec1 = ResBlock(base_ch + base_ch, base_ch, tdim)
self.out_conv = nn.Conv2d(base_ch, out_ch, kernel_size=3, padding=1)

def forward(self, x: torch.Tensor, t: torch.Tensor) -> torch.Tensor:
    temb = self.time_mlp(self.time(t)) # (B, tdim)

    x = self.in_conv(x)
    s1 = self.enc1(x, temb) # skip
    h = self.down(s1)
    h = self.mid(h, temb)
    h = self.up(h)
    h = torch.cat([h, s1], dim=1) # skip connection
    h = self.dec1(h, temb)
    out = self.out_conv(h)
    return out

# Smoke test: forward pass shapes
B, C, H, W = 4, 1, 32, 32
x = torch.randn(B, C, H, W, device=device)
t = torch.randint(0, 200, (B,), device=device)
net2d = TinyUNet2D(in_ch=C, base_ch=32, tdim=128).to(device)

y = net2d(x, t)
print("input:", x.shape, "output:", y.shape)

```

```
input: torch.Size([4, 1, 32, 32]) output: torch.Size([4, 1, 32, 32])
```

✓ Tiny training example on synthetic 2D blob images

What this example demonstrates:

- how to create a simple synthetic **blob dataset** (32×32 images),
- how to train a tiny U-Net to predict noise $\epsilon_\theta(x_t, t)$,
- how to visualize **clean** vs **noisy** vs **true noise** vs **predicted noise**.

Important: This cell assumes `TinyUNet2D`, `DiffusionSchedule`, and `Diffusion2D` were defined earlier in the notebook.

```
import numpy as np
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

assert "TinyUNet2D" in globals(), "TinyUNet2D must be defined earlier"
assert "DiffusionSchedule" in globals() and "Diffusion2D" in globals()

# --- synthetic blob images ---
def render_gaussian_blob(H, W, cx, cy, sigma):
    ys = np.arange(H)[:, None]
    xs = np.arange(W)[None, :]
    g = np.exp(-((xs-cx)**2 + (ys-cy)**2) / (2*sigma**2))
    return g.astype(np.float32)

def sample_blob_images(batch=64, H=32, W=32, device=device):
    imgs = []
    for _ in range(batch):
        img = np.zeros((H, W), dtype=np.float32)
        k = np.random.randint(1, 3) # 1 or 2 blobs
        for _ in range(k):
            cx = np.random.uniform(8, W-8)
            cy = np.random.uniform(8, H-8)
            sigma = np.random.uniform(1.5, 4.0)
            amp = np.random.uniform(0.6, 1.0)
            img += amp * render_gaussian_blob(H, W, cx, cy, sigma)
        imgs.append(np.clip(img, 0.0, 1.0))
    x0 = torch.from_numpy(np.stack(imgs)).to(device).unsqueeze(1)
    return x0
```

```

# --- model + diffusion wrapper ---
net2d = TinyUNet2D(in_ch=1, base_ch=32, tdim=128).to(device)
opt2d = torch.optim.AdamW(net2d.parameters(), lr=2e-4)

sched2d = DiffusionSchedule(T=200, device=device)
diff2d = Diffusion2D(net2d, sched2d)

# --- training loop (short demo) ---
losses2d = []
for it in range(300):
    x0 = sample_blob_images(batch=64, H=32, W=32)
    loss = diff2d.train_step(x0, opt2d)
    losses2d.append(loss)
    if (it+1) % 50 == 0:
        print(f"iter {it+1}/300 | loss {np.mean(losses2d[-50:]):.4f}")

plt.figure(figsize=(6,3))
plt.plot(losses2d)
plt.title("2D blob demo: training loss")
plt.xlabel("iteration")
plt.ylabel("MSE")
plt.tight_layout()
plt.show()

# --- visualization (NO backward here) ---
@torch.no_grad()
def show_row(tensors, titles):
    plt.figure(figsize=(12,3))
    for i, (ten, title) in enumerate(zip(tensors, titles)):
        ax = plt.subplot(1, len(tensors), i+1)
        img = ten[0,0].detach().cpu().numpy()
        vmax = 1.0 if title.startswith("x0") else None
        ax.imshow(img, cmap="gray", vmin=0.0 if vmax is not None else 1.0)
        ax.set_title(title)
        ax.axis("off")
    plt.tight_layout()
    plt.show()

with torch.no_grad():
    x0 = sample_blob_images(batch=8, H=32, W=32)
    loss, x0, xt, eps, eps_pred, t = diff2d.eval_step(x0)

# scale noise images for display only
eps_s = (eps - eps.min())/(eps.max()-eps.min()+1e-8)
eps_p_s = (eps_pred - eps_pred.min())/(eps_pred.max()-eps_pred.min())

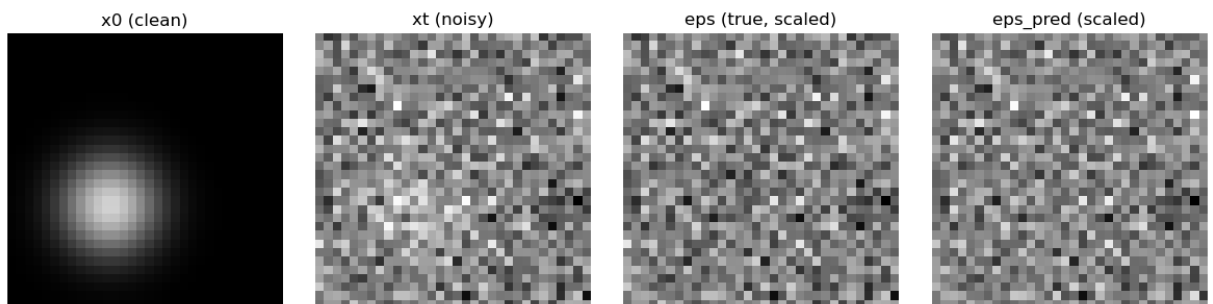
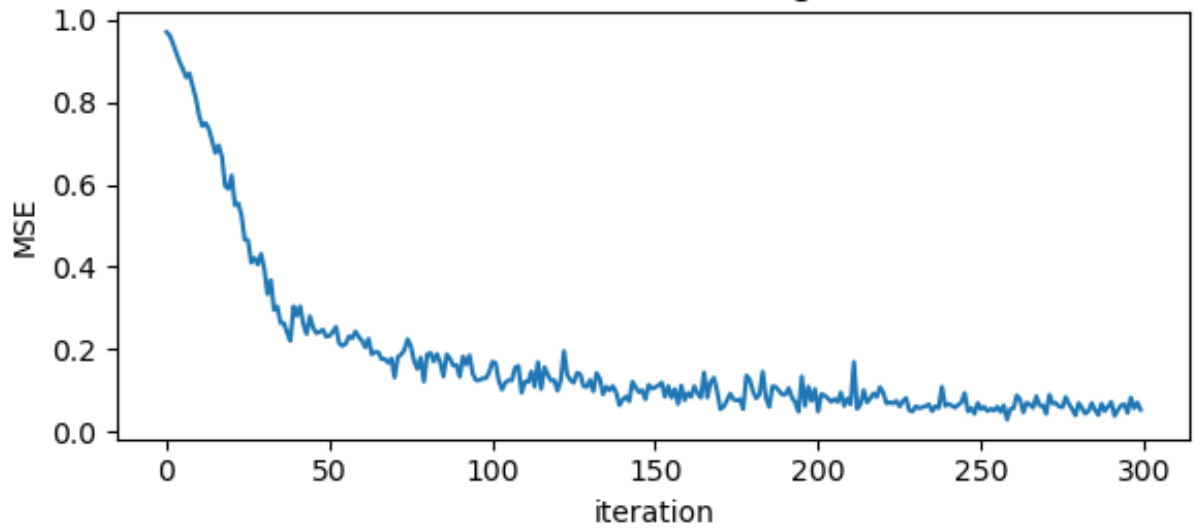
show_row([x0, xt, eps_s, eps_p_s],

```

```
["x0 (clean)", "xt (noisy)", "eps (true, scaled)", "eps_pr
```

```
iter 50/300 | loss 0.5256
iter 100/300 | loss 0.1827
iter 150/300 | loss 0.1190
iter 200/300 | loss 0.0934
iter 250/300 | loss 0.0723
iter 300/300 | loss 0.0595
```

2D blob demo: training loss



✓ Visualize the dataset and the reverse process (generation)

We now:

1. Show a small batch of **clean blob images** x_0 (the training distribution).
2. Run the **reverse DDPM chain** starting from pure noise $x_T \sim \mathcal{N}(0, I)$.
3. Visualize a **denoising trajectory** (several intermediate x_t 's) so you can see noise turning into blobs.

```
import numpy as np
import torch
import matplotlib.pyplot as plt

# ---- 1) show some clean blob samples (x0) ----
@torch.no_grad()
def show_grid(x, title, nrow=4):
    B = x.shape[0]
    n = min(B, nrow*nrow)
    plt.figure(figsize=(nrow*2, nrow*2))
    for i in range(n):
        plt.subplot(nrow, nrow, i+1)
        plt.imshow(x[i,0].detach().cpu().numpy(), cmap="gray", vmin=0, vmax=1)
        plt.axis("off")
    plt.suptitle(title)
    plt.tight_layout()
    plt.show()

with torch.no_grad():
    x0_batch = sample_blob_images(batch=16, H=32, W=32)
    show_grid(x0_batch, "Clean samples from the 2D blob dataset (x0)",

# ---- 2) reverse DDPM generation with a trajectory ----
@torch.no_grad()
def sample_ddpm_with_trajectory(diff, shape, snap_ts):
    # Returns final sample and a dict {t: x_t} for selected timesteps
    device = diff.sched.device
    xt = torch.randn(*shape, device=device) # x_T
    snaps = {}
    T = diff.sched.T
    snap_set = set(snap_ts)
    for time in range(T-1, -1, -1):
        t = torch.full((shape[0],), time, device=device, dtype=torch.long)
        xt = diff.ddpm_step(xt, t)
        if time in snap_set:
            snaps[time] = xt.detach().clone()
```



```

    return xt, snaps

snap_ts = (sched2d.T-1,
           int(0.75*(sched2d.T-1)),
           int(0.5*(sched2d.T-1)),
           int(0.3*(sched2d.T-1)),
           int(0.15*(sched2d.T-1)),
           5, 0)

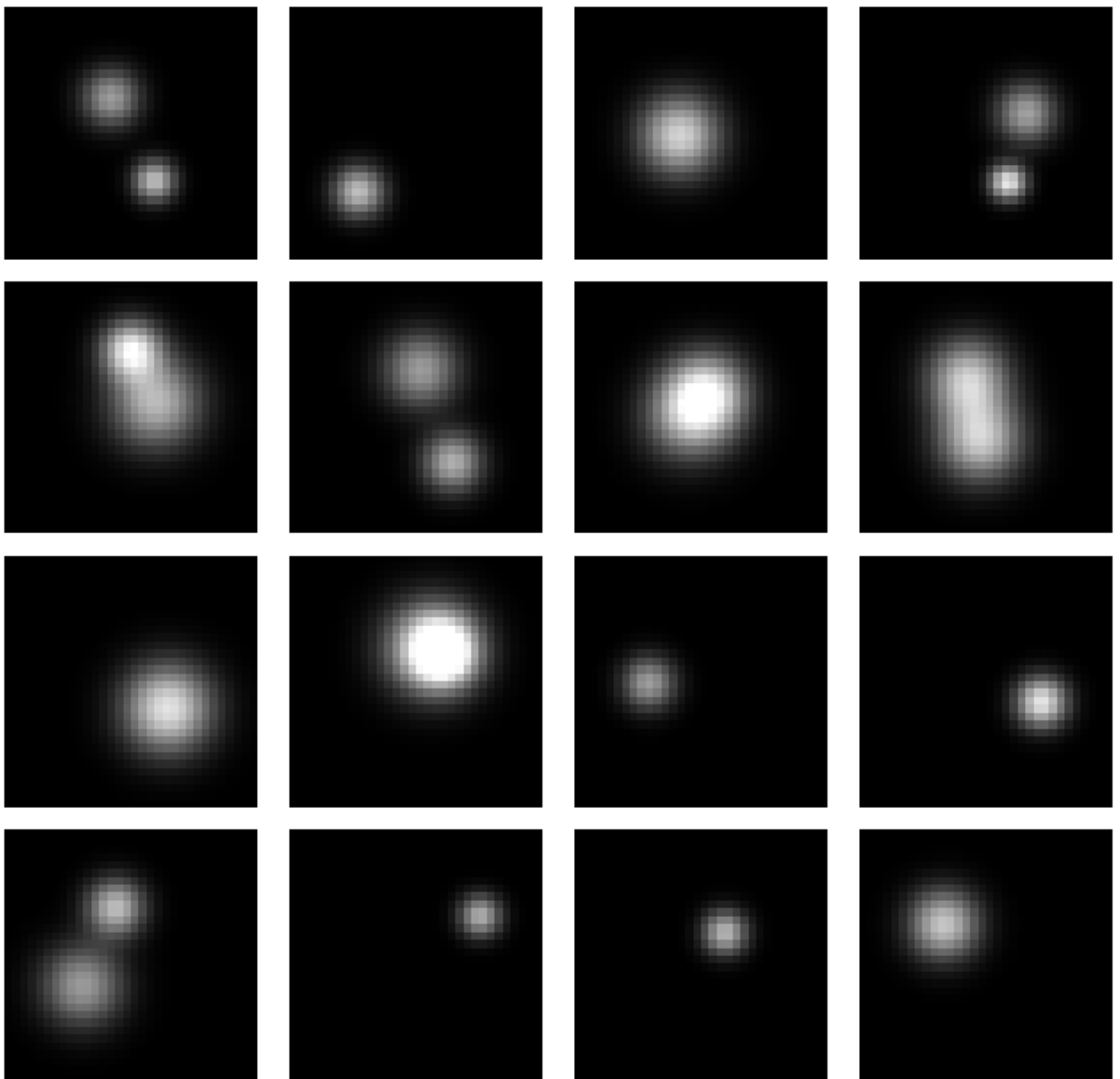
with torch.no_grad():
    x_final, snaps = sample_ddpm_with_trajectory(diff2d, shape=(16,

for t_show in sorted(snaps.keys(), reverse=True):
    show_grid(snaps[t_show].clamp(0,1), f"Denoising snapshot at t={

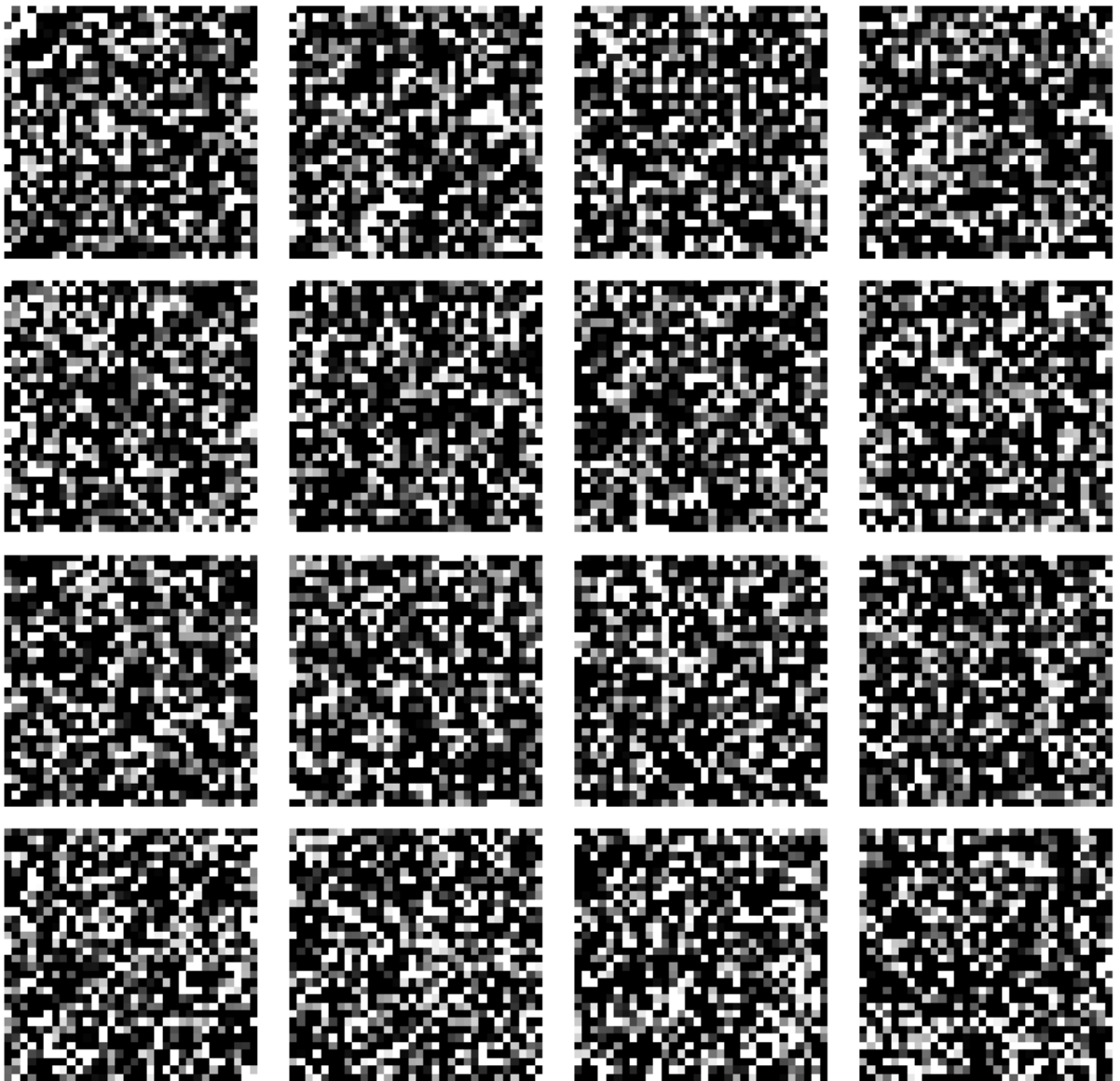
show_grid(x_final.clamp(0,1), "Final generated blobs (DDPM reverse

```

Clean samples from the 2D blob dataset (x0)



Denoising snapshot at t=199

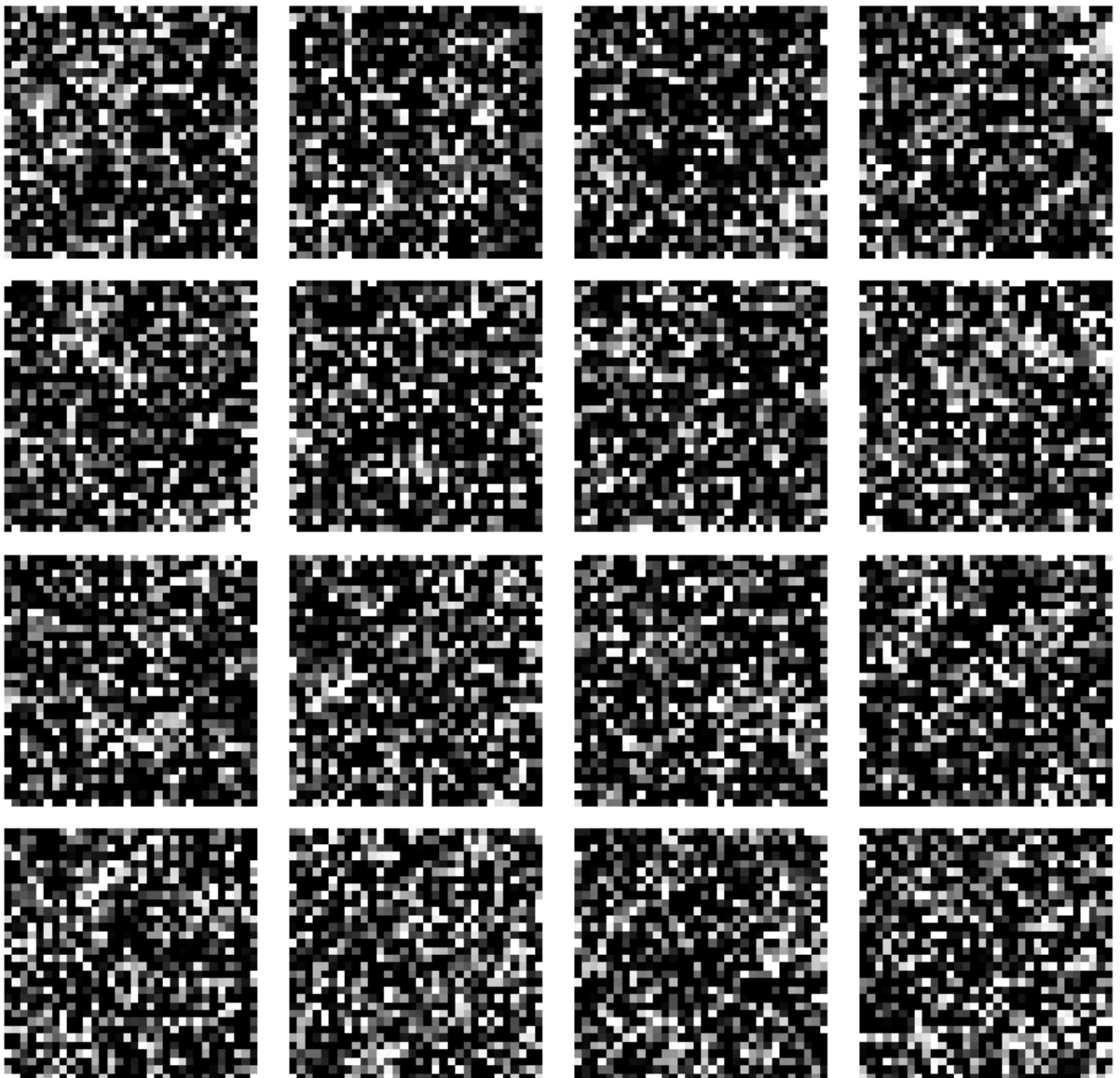


Denoising snapshot at t=149

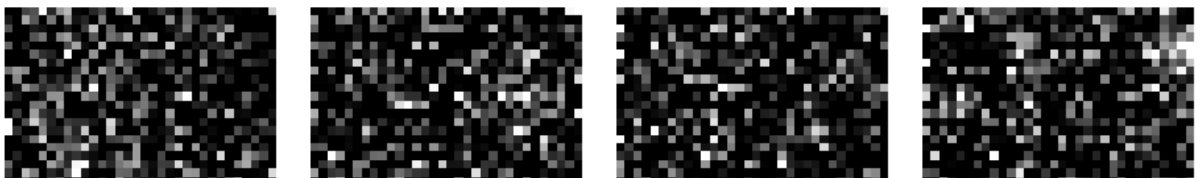


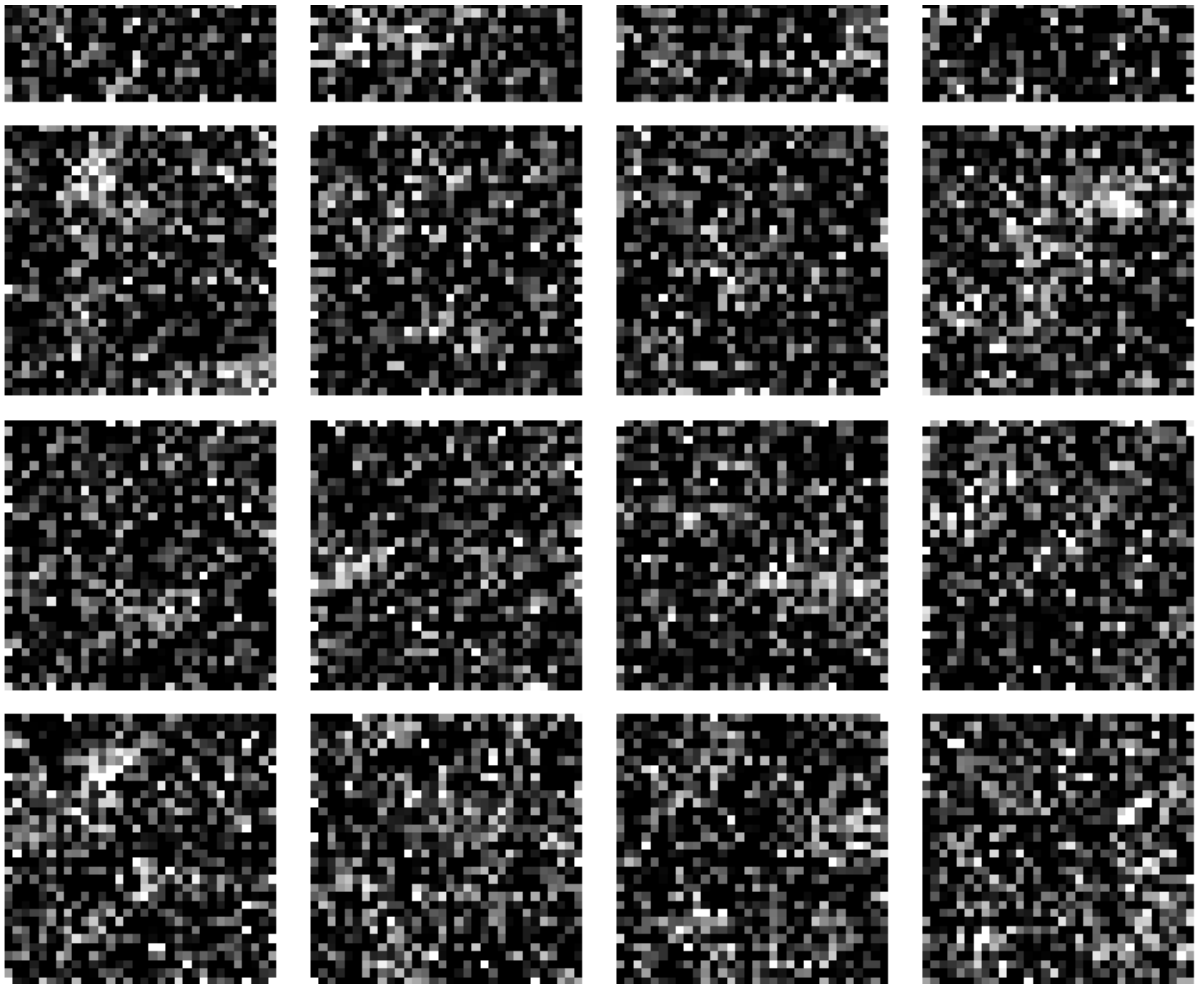
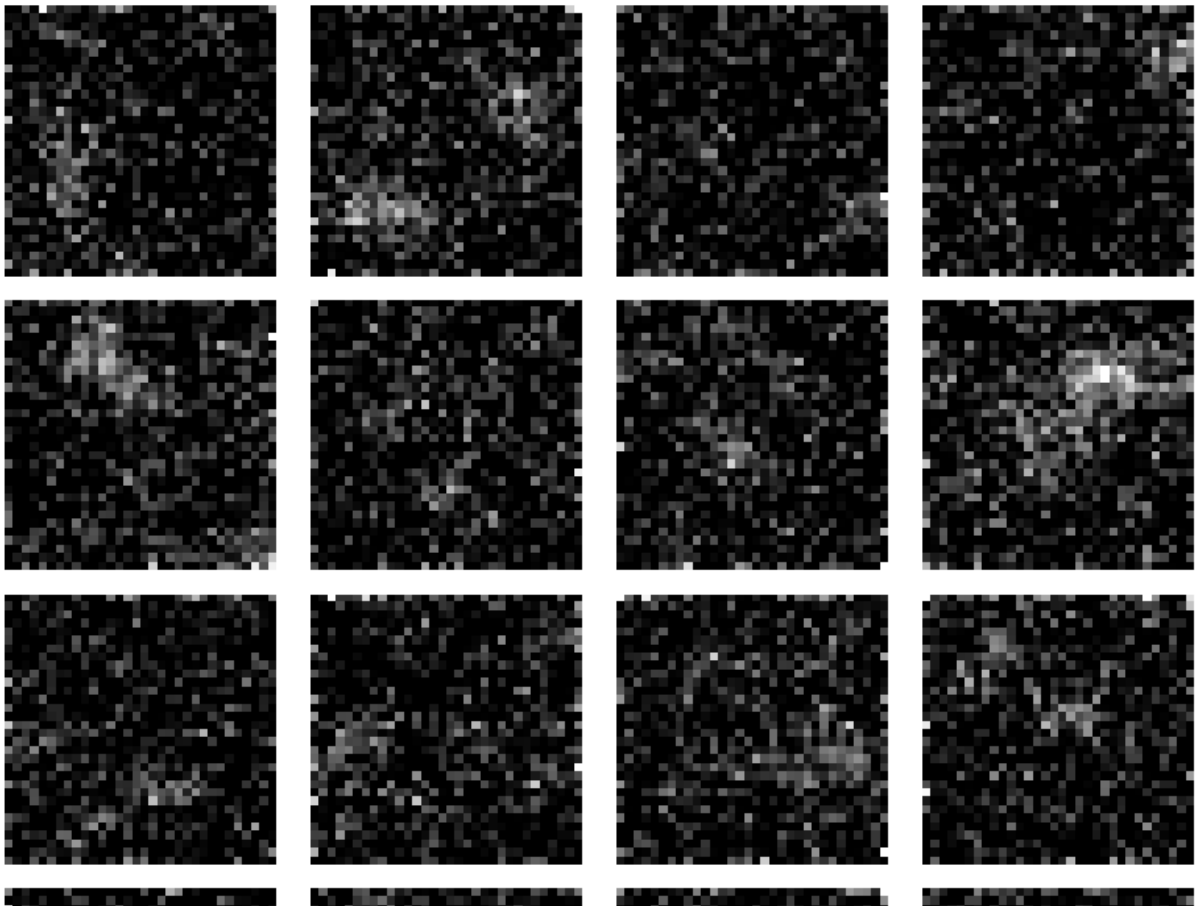


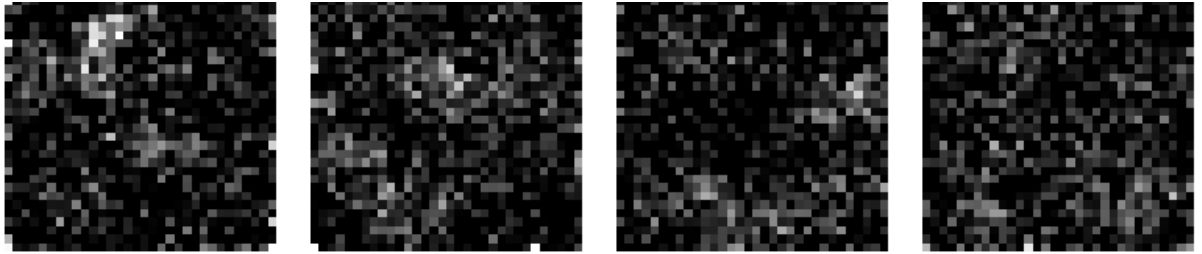
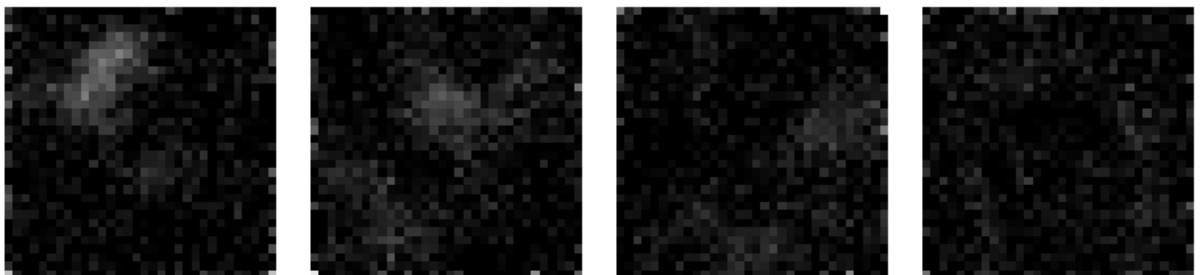
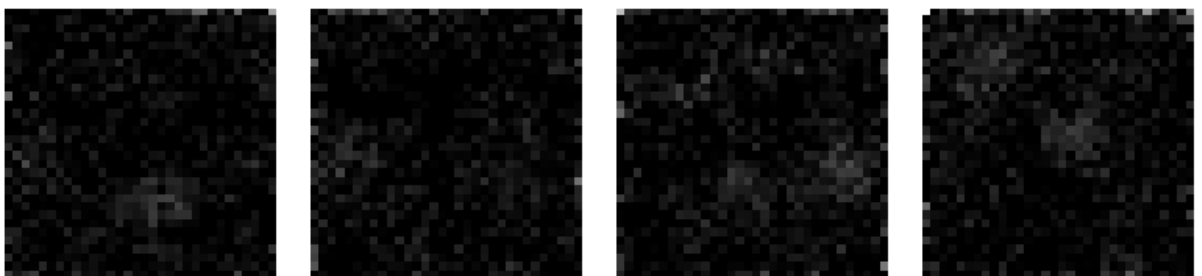
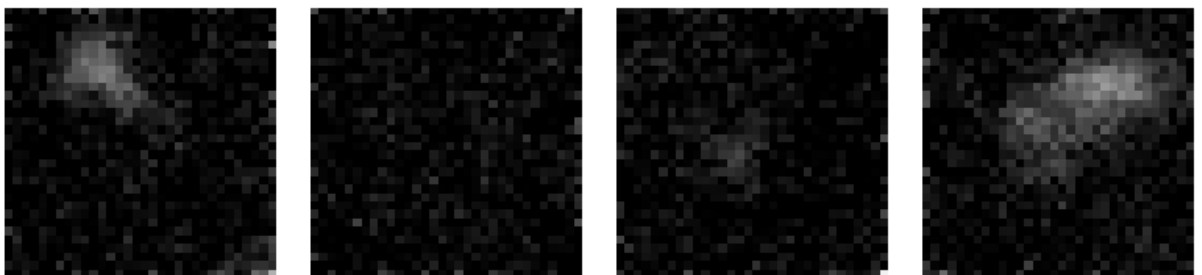
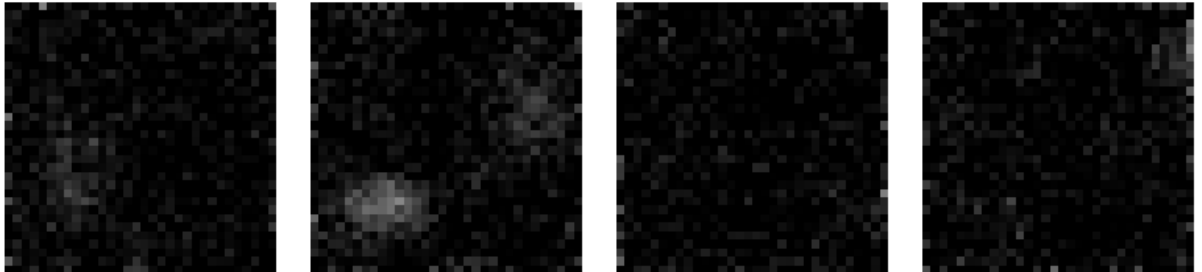
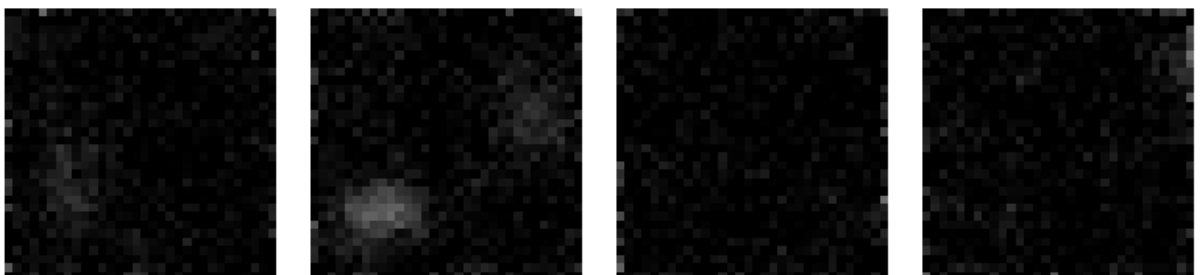
Denoising snapshot at t=99

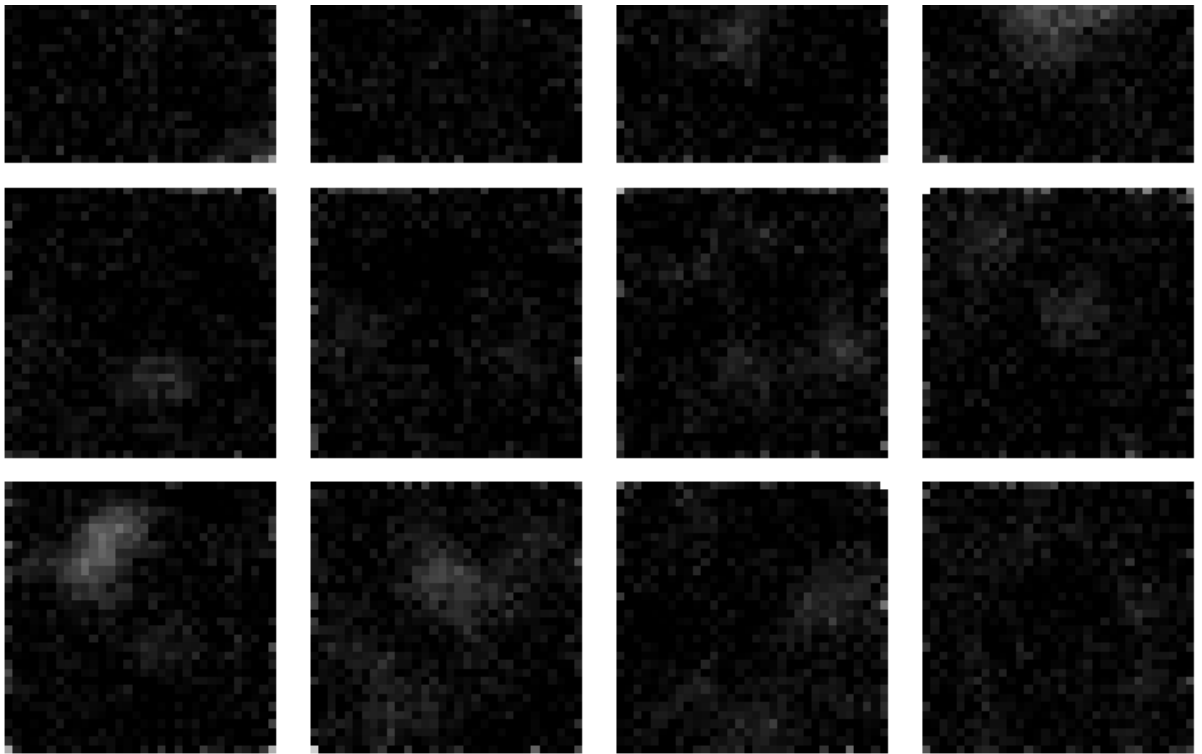


Denoising snapshot at t=59

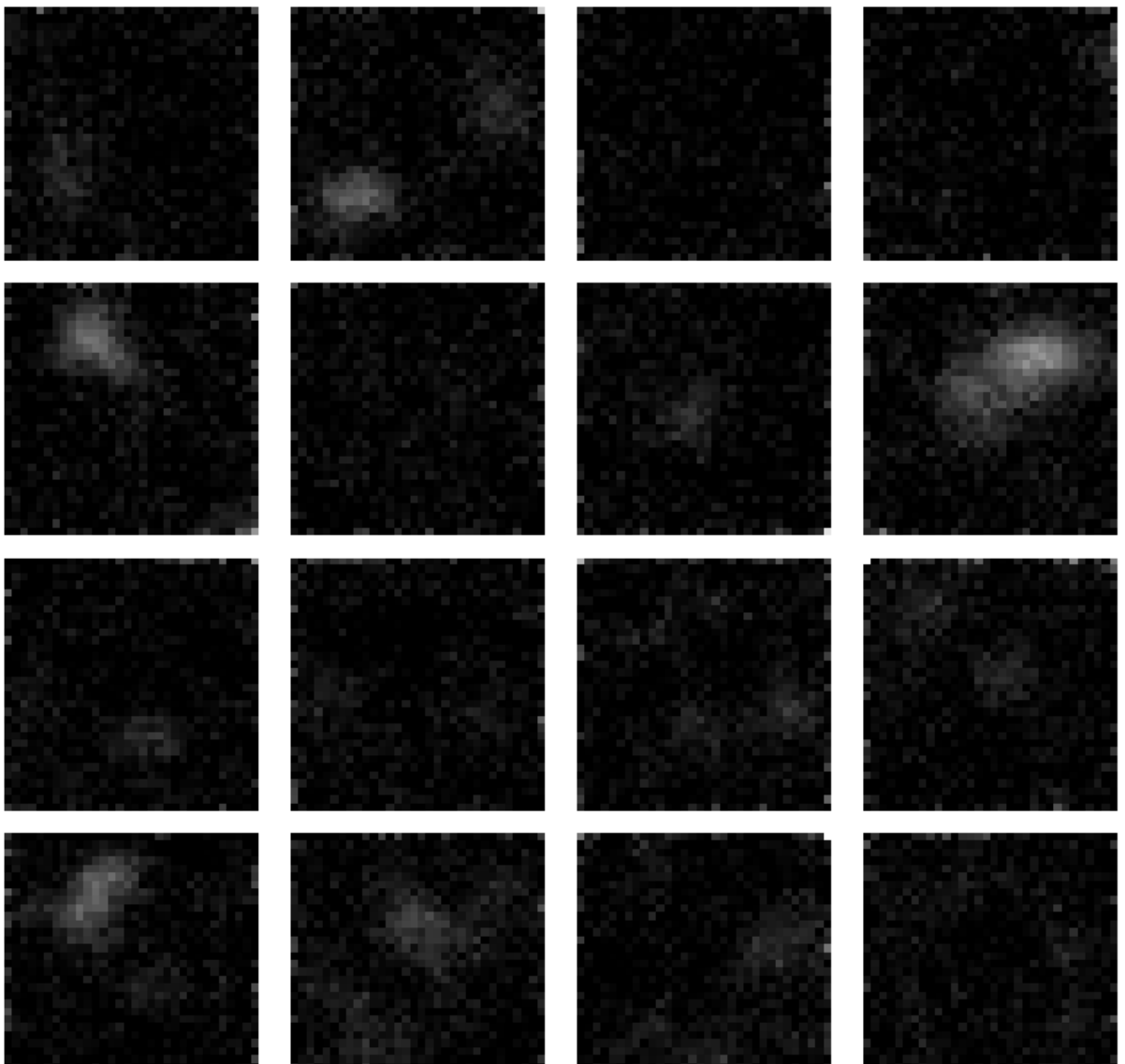


Denoising snapshot at $t=29$ 

Denoising snapshot at $t=5$ Denoising snapshot at $t=0$ 



Final generated blobs (DDPM reverse chain)



✓ DDIM (Denoising Diffusion Implicit Models)

DDIM is best understood as a **sampling method** for a model trained like DDPM.

- **Training:** unchanged (still predict noise with an MSE loss)

$$\mathbb{E} \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2.$$

- **Sampling:** choose a different reverse update that can be **deterministic** (often much faster).

Key idea

From the forward marginal

\$\$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon,$$

\$\$

we can estimate x_0 from (x_t, t) using the network's ϵ prediction:

\$\$

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

\$\$

DDIM proposes a family of reverse updates (indexed by a parameter $\eta \in [0, 1]$):

\$\$

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_{\theta}(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

\$\$

with

\$\$

$$\sigma_t(\eta) = \eta, \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}}.$$

\$\$

- $\eta = 1$ gives a stochastic sampler similar in spirit to DDPM.
- $\eta = 0$ **gives a deterministic sampler** (no added noise), often used to reduce steps.

Below is a minimal **1D example** comparing DDPM-style sampling vs DDIM sampling (deterministic and stochastic).

```
import math
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(0)
np.random.seed(0)

# -----
# 1) 1D toy data: two bumps
# -----
def sample_x0(n, sigma_data=0.35, device=device):
    comp = torch.randint(0, 2, (n,), device=device)
    means = torch.where(comp == 0, torch.tensor(-2.0, device=device),
                        torch.tensor(2.0, device=device))
    x0 = means + sigma_data * torch.randn(n, device=device)
    return x0.view(-1, 1) # (n,1)

# -----
# 2) Schedule (VP / DDPM style)
# -----
def make_linear_beta_schedule(T, beta_start=1e-4, beta_end=0.02, device=device):
    betas = torch.linspace(beta_start, beta_end, T, device=device)
    alphas = 1.0 - betas
    alpha_bars = torch.cumprod(alphas, dim=0)
    return betas, alphas, alpha_bars

T = 200
betas, alphas, alpha_bars = make_linear_beta_schedule(T)

def extract(a, t, x_shape):
    out = a.gather(0, t)
```

```

        return out.view(-1, *([1]*(len(x_shape)-1)))

def q_sample(x0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x0)
    a_bar = extract(alpha_bars, t, x0.shape)
    xt = torch.sqrt(a_bar)*x0 + torch.sqrt(1.0-a_bar)*noise
    return xt, noise

# -----
# 3) Tiny epsilon model (1D) with sinusoidal time embedding
# -----
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dim = dim
    def forward(self, t):
        half = self.dim // 2
        freqs = torch.exp(-math.log(10000) * torch.arange(0, half,
            args = t.float().unsqueeze(1) * freqs.unsqueeze(0)
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=1)
        if self.dim % 2 == 1:
            emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], di
        return emb

class EpsMLP1D(nn.Module):
    def __init__(self, tdim=64, hidden=128):
        super().__init__()
        self.tenc = SinusoidalTimeEmbedding(tdim)
        self.net = nn.Sequential(
            nn.Linear(1 + tdim, hidden),
            nn.SiLU(),
            nn.Linear(hidden, hidden),
            nn.SiLU(),
            nn.Linear(hidden, 1),
        )
    def forward(self, x, t):
        temb = self.tenc(t)
        return self.net(torch.cat([x, temb], dim=1))

eps_model = EpsMLP1D().to(device)
opt = torch.optim.AdamW(eps_model.parameters(), lr=2e-4)

# Quick training for demo
steps = 2000
batch = 512
eps_model.train()
for step in range(1, steps+1):

```

```

    x0 = sample_x0(batch)
    t = torch.randint(0, T, (batch,), device=device)
    xt, eps = q_sample(x0, t)
    eps_pred = eps_model(xt, t)
    loss = F.mse_loss(eps_pred, eps)
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 500 == 0:
        print(f"train step {step}/{steps} | loss {loss.item():.4f}")

# -----
# 4) DDPM sampling (baseline)
# -----
@torch.no_grad()
def ddpm_step(xt, t):
    beta_t = extract(betas, t, xt.shape)
    alpha_t = extract(alphas, t, xt.shape)
    a_bar_t = extract(alphaBars, t, xt.shape)

    eps_pred = eps_model(xt, t)
    mean = (1.0/torch.sqrt(alpha_t)) * (xt - (beta_t/torch.sqrt(1.0 - alpha_t)) * eps_pred)

    if (t == 0).all():
        return mean
    noise = torch.randn_like(xt)
    return mean + torch.sqrt(beta_t) * noise

@torch.no_grad()
def sample_ddpm(n=60000):
    xt = torch.randn(n, 1, device=device)
    for time in reversed(range(T)):
        t = torch.full((n,), time, device=device, dtype=torch.long)
        xt = ddpm_step(xt, t)
    return xt.squeeze(1).cpu().numpy()

# -----
# 5) DDIM sampling (eta controls stochasticity)
# -----
@torch.no_grad()
def ddim_step(xt, t, eta=0.0):
    """
    DDIM update:
    x0_hat = (xt - sqrt(1-a_bar_t)*eps)/sqrt(a_bar_t)
    x_{t-1} = sqrt(a_bar_{t-1})*x0_hat + sqrt(1-a_bar_{t-1}-sigma**2)*eps
    """
    a_bar_t = extract(alphaBars, t, xt.shape)
    eps_pred = eps_model(xt, t)

```

```

x0_hat = (xt - torch.sqrt(1.0 - a_bar_t) * eps_pred) / torch.sq

# a_bar_{t-1} (handle t=0)
t_prev = torch.clamp(t-1, min=0)
a_bar_prev = extract(alphaBars, t_prev, xt.shape)

# sigma_t(eta)
eps = 1e-12
frac = (1.0 - a_bar_prev) / (1.0 - a_bar_t + eps)
inside = 1.0 - (a_bar_t / (a_bar_prev + eps))
sigma_t = eta * torch.sqrt(frac * torch.clamp(inside, min=0.0))

dir_coeff = torch.sqrt(torch.clamp(1.0 - a_bar_prev - sigma_t**2, min=0.0))
mean = torch.sqrt(a_bar_prev) * x0_hat + dir_coeff * eps_pred

if (t == 0).all():
    return mean
z = torch.randn_like(xt)
return mean + sigma_t * z

@torch.no_grad()
def sample_ddim(n=60000, eta=0.0, stride=1):
    """
    stride>1 demonstrates fewer steps (e.g., stride=2 uses half the
    """
    xt = torch.randn(n, 1, device=device)
    times = list(range(T-1, -1, -stride))
    for time in times:
        t = torch.full((n,), time, device=device, dtype=torch.long)
        xt = ddim_step(xt, t, eta=eta)
    return xt.squeeze(1).cpu().numpy()

# Generate samples
x_ddpm = sample_ddpm(n=60000)
x_ddim_det = sample_ddim(n=60000, eta=0.0, stride=1) # determini
x_ddim_sto = sample_ddim(n=60000, eta=1.0, stride=1) # stochasti

# Plot histograms
bins = np.linspace(-5, 5, 200)
plt.figure(figsize=(10,3))
plt.hist(x_ddpm, bins=bins, density=True, alpha=0.5, label="DDPM (s
plt.hist(x_ddim_det, bins=bins, density=True, alpha=0.5, label="DDI
plt.hist(x_ddim_sto, bins=bins, density=True, alpha=0.5, label="DDI
plt.axvline(0, linestyle="--")
plt.title("1D sampling comparison: DDPM vs DDIM")
plt.xlabel("x")
plt.ylabel("density")

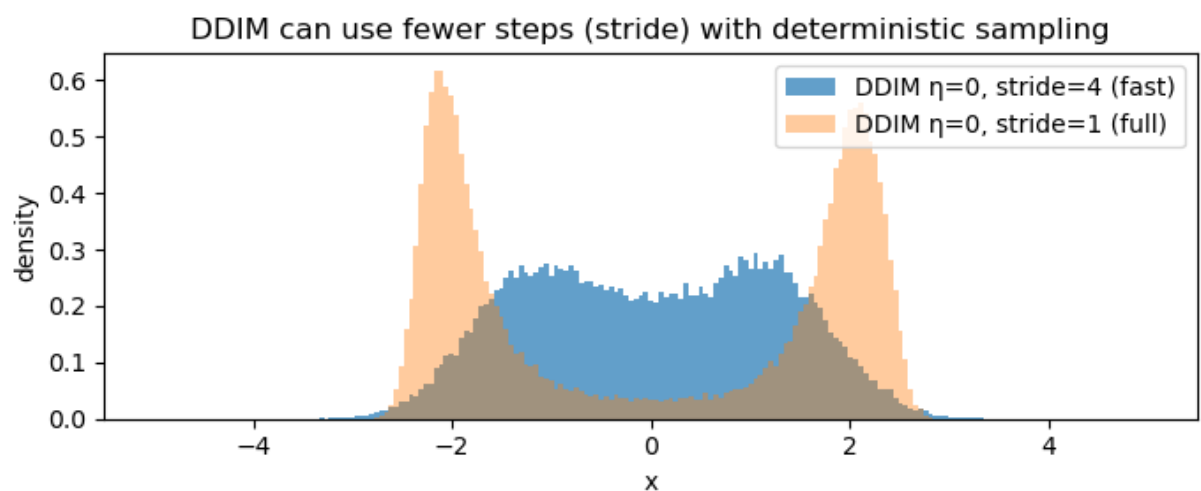
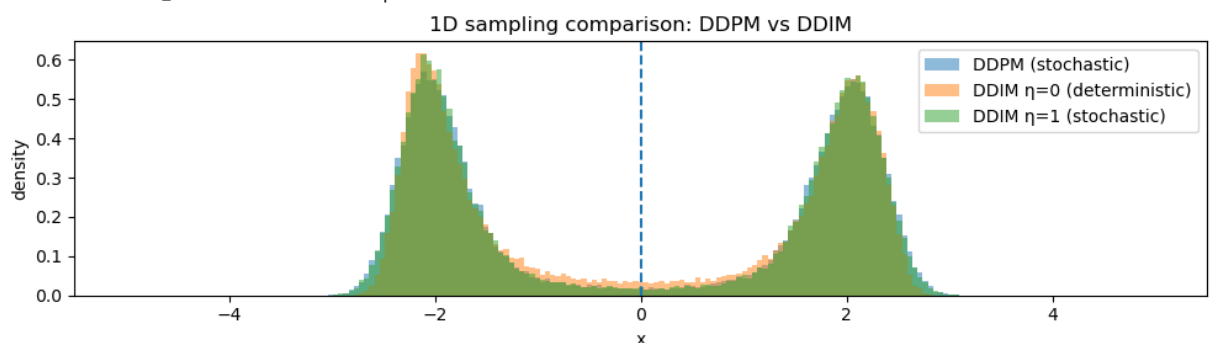
```

```
plt.legend()
plt.tight_layout()
plt.show()

# Show the effect of fewer steps (stride)
x_ddim_fast = sample_ddim(n=60000, eta=0.0, stride=4) # 4x fewer
plt.figure(figsize=(7,3))
plt.hist(x_ddim_fast, bins=bins, density=True, alpha=0.7, label="DDIM")
plt.hist(x_ddim_det, bins=bins, density=True, alpha=0.4, label="DDIM")
plt.title("DDIM can use fewer steps (stride) with deterministic sampling")
plt.xlabel("x")
plt.ylabel("density")
plt.legend()
plt.tight_layout()
plt.show()

print("Takeaway: DDIM changes the reverse update (sampling), not the training")
```

```
train step 500/2000 | loss 0.8042
train step 1000/2000 | loss 0.7831
train step 1500/2000 | loss 0.6648
train step 2000/2000 | loss 0.6016
```



Takeaway: DDIM changes the reverse update (sampling), not the training

✓ Animating forward and backwards Diffusion Processes

```
# 1D Forward Diffusion Animation (fixed y-scale + text moved below

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# ----- Settings -----
T = 80
x0 = 3.0

beta_min, beta_max = 0.005, 0.08
betas = np.linspace(beta_min, beta_max, T)
alphas = 1.0 - betas

grid = np.linspace(-6, 6, 600)

def normal_pdf(x, mu, var):
    return (1.0 / np.sqrt(2*np.pi*var)) * np.exp(-(x-mu)**2 / (2*va

# ----- Simulate one forward trajectory -----
xs = [x0]
mus, vars_ = [], []
eps = np.random.randn(T)

x_prev = x0
for t in range(T):
    a = alphas[t]
    mu = np.sqrt(a) * x_prev
    var = 1.0 - a
    x_t = mu + np.sqrt(var) * eps[t]
    mus.append(mu)
    vars_.append(var)
    xs.append(x_t)
    x_prev = x_t

xs = np.array(xs)
```

```

# ----- Precompute a stable y-limit -----
peaks = np.array([1.0 / np.sqrt(2*np.pi*v) for v in vars_])
y_max = 1.15 * np.percentile(peaks, 95)
y_max = max(y_max, 0.5)

# ----- Plot + animation -----
fig, ax = plt.subplots(figsize=(8, 4))

# Make room at the bottom for text (so it doesn't overlap the plot)
fig.subplots_adjust(bottom=0.30)

line_pdf, = ax.plot([], [], lw=2, label=r"$q(x_t \mid x_{t-1})$")
dot_prev = ax.scatter([], [], s=60, label=r"$x_{t-1}$")
dot_mean = ax.scatter([], [], s=60, label=r"mean $\sqrt{\alpha_t}x_0$")
dot_sample = ax.scatter([], [], s=80, label=r"sample $x_t$")

ax.set_xlim(grid.min(), grid.max())
ax.set_ylim(0, y_max)
ax.set_xlabel("x")
ax.set_ylabel("density")
ax.set_title("Forward diffusion in 1D: sample from a Gaussian around")
ax.legend(loc="upper right")

# Put the text BELOW the axes in figure coordinates
text_obj = fig.text(
    0.02, 0.02, "", ha="left", va="bottom", fontsize=10, family="mc
)

def init():
    line_pdf.set_data([], [])
    dot_prev.set_offsets(np.empty((0, 2)))
    dot_mean.set_offsets(np.empty((0, 2)))
    dot_sample.set_offsets(np.empty((0, 2)))
    text_obj.set_text("")
    return line_pdf, dot_prev, dot_mean, dot_sample, text_obj

def update(frame):
    t = frame - 1
    x_prev = xs[frame-1]
    x_curr = xs[frame]
    mu = mus[t]
    var = vars_[t]

    pdf = normal_pdf(grid, mu, var)
    line_pdf.set_data(grid, pdf)

    # points near x-axis for visibility

```



```

y0 = 0.02 * y_max
dot_prev.set_offsets(np.array([[x_prev, y0]]))
dot_mean.set_offsets(np.array([[mu, y0]]))
dot_sample.set_offsets(np.array([[x_curr, y0]]))

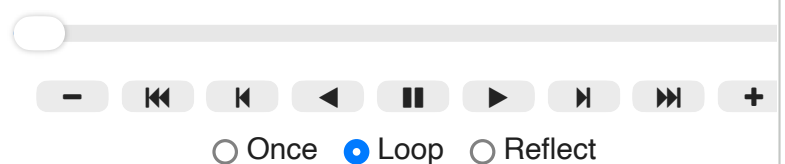
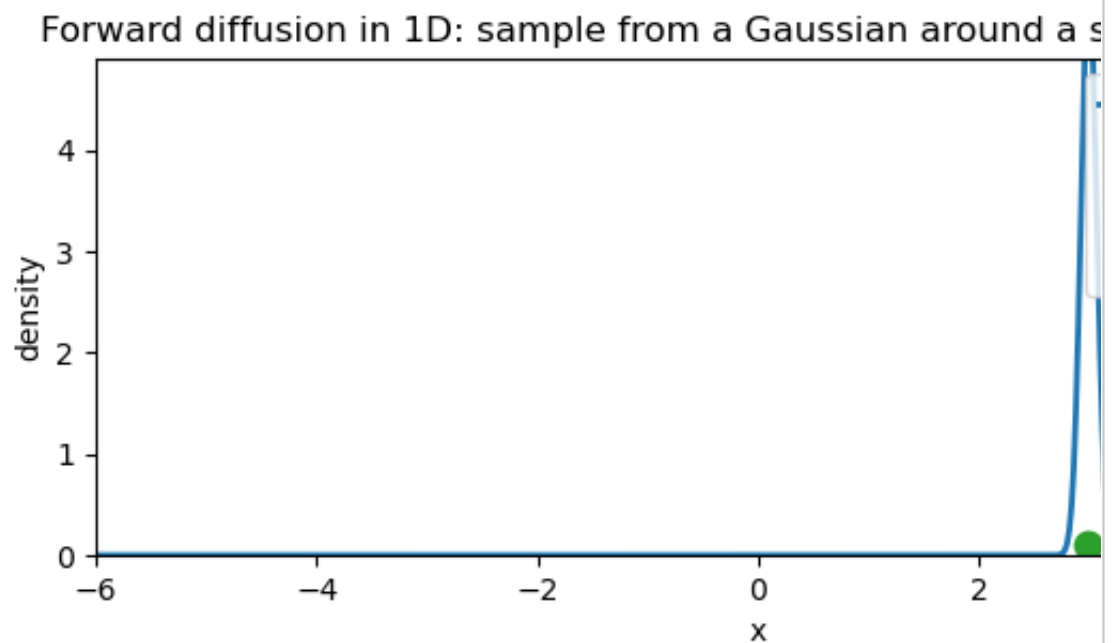
text_obj.set_text(
    f"t={frame:02d} | alpha_t={alphas[t]:.4f} beta_t={betas[t]:.4f} | "
    f"mean = sqrt(alpha_t)*x_(t-1) = {mu:.3f} | var = 1-alpha_t"
    f"x_(t-1) = {x_prev:.3f} -> x_t = {x_curr:.3f}"
)

return line_pdf, dot_prev, dot_mean, dot_sample, text_obj

anim = FuncAnimation(fig, update, frames=np.arange(1, T+1), init_func=init)

plt.close(fig)
HTML(anim.to_jshtml())

```



```

# Self-contained: Learn 1D DDPM reverse + ANIMATE the reverse condition
# What you will see:

```

```

# - Start at  $x_T \sim N(0,1)$ 
# - For each reverse step  $t$ : show  $p_{\theta}(x_{t-1} | x_t) = N(\mu_{\theta}, \sigma_{\theta}^2)$ 
# - Plot the Gaussian PDF at each step (this is the "reverse process")
# - Show the current point  $x_t$ , the mean  $\mu_{\theta}$ , and the sampled  $x_{t-1}$ 
# - Show SNR( $t$ ) as context
#
# VS Code / Jupyter friendly.

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# -----
# 0) Repro + device
# -----
seed = 0
np.random.seed(seed)
torch.manual_seed(seed)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

# -----
# 1) Data: 1D two-bump distribution
# -----
def sample_two_bumps(batch, sigma=0.35, m1=-2.0, m2=2.0, device=device):
    comp = torch.randint(0, 2, (batch,), device=device)
    means = torch.where(comp == 0, torch.tensor(m1, device=device),
                        torch.tensor(m2, device=device))
    return means + sigma * torch.randn(batch, device=device) # (B, 1)

# -----
# 2) Diffusion schedule (DDPM)
# -----
T = 200
beta_min, beta_max = 1e-4, 0.02
betas = torch.linspace(beta_min, beta_max, T, device=device) # (T, 1)
alphas = 1.0 - betas # (T, 1)
alpha_bars = torch.cumprod(alphas, dim=0) # (T, 1)

def snr_t(t_idx: int) -> float:
    ab = float(alpha_bars[t_idx].detach().cpu().item())
    return ab / max(1e-12, (1.0 - ab))

def snr_db(snr: float) -> float:
    return 10.0 * np.log10(max(1e-12, snr))

```

```

def q_sample(x0, t, noise=None):
    # x0: (B,), t: (B,)
    if noise is None:
        noise = torch.randn_like(x0)
    ab = alpha_bars[t]
    return torch.sqrt(ab) * x0 + torch.sqrt(1.0 - ab) * noise, noise

def posterior_variance(t_idx: int):
    # \tilde{\beta}_t = \beta_t * (1 - \bar{\alpha}_{t-1}) / (1 - \bar{\alpha}_t)
    if t_idx == 0:
        return torch.tensor(0.0, device=device)
    bt = betas[t_idx]
    ab_t = alpha_bars[t_idx]
    ab_tm1 = alpha_bars[t_idx - 1]
    return bt * (1.0 - ab_tm1) / (1.0 - ab_t)

def p_mean_from_eps(xt, t_idx: int, eps_pred):
    # \mu_\theta = 1/\sqrt{\alpha_t} * (x_t - \beta_t/\sqrt{1-\alpha_t} * \epsilon)
    at = alphas[t_idx]
    bt = betas[t_idx]
    ab_t = alpha_bars[t_idx]
    return (1.0 / torch.sqrt(at)) * (xt - (bt / torch.sqrt(1.0 - ab_t)) * eps_pred)

# -----
# 3) Sinusoidal time embedding
# -----
def sinusoidal_t_embed(t, dim=64):
    # t: (B,) int64
    t = t.float().unsqueeze(1) # (B,1)
    half = dim // 2
    freqs = torch.exp(
        -np.log(10000.0) * torch.arange(0, half, device=device).float()
    ) # (half,)
    angles = t * freqs.unsqueeze(0) # (B,half)
    return torch.cat([torch.sin(angles), torch.cos(angles)], dim=1)

# -----
# 4) Tiny epsilon network eps_theta(x_t, t) for scalars
# -----
class EpsNet1D(nn.Module):
    def __init__(self, tdim=64, hidden=128):
        super().__init__()
        self.fc1 = nn.Linear(1 + tdim, hidden)
        self.fc2 = nn.Linear(hidden, hidden)
        self.fc3 = nn.Linear(hidden, 1)

    def forward(self, xt, t):

```

```

        xt = xt.unsqueeze(1)                # (B,1)
        te = sinusoidal_t_embed(t, dim=64)  # (B,64)
        h = torch.cat([xt, te], dim=1)
        h = F.silu(self.fc1(h))
        h = F.silu(self.fc2(h))
        return self.fc3(h).squeeze(1)       # (B,)

net = EpsNet1D(tdim=64, hidden=128).to(device)
opt = torch.optim.AdamW(net.parameters(), lr=2e-3)

# -----
# 5) Train quickly (DDPM noise prediction)
# -----
def train_step(batch=1024):
    x0 = sample_two_bumps(batch)
    t = torch.randint(0, T, (batch,), device=device)
    xt, eps = q_sample(x0, t)
    eps_pred = net(xt, t)
    loss = F.mse_loss(eps_pred, eps)
    opt.zero_grad()
    loss.backward()
    opt.step()
    return float(loss.item())

steps = 1200 # small demo
losses = []
for it in range(steps):
    losses.append(train_step())
    if (it + 1) % 300 == 0:
        print(f"iter {it+1:4d}/{steps} | loss {losses[-1]:.4f}")

plt.figure(figsize=(6,3))
plt.plot(losses)
plt.title("Training loss: MSE( eps_pred, eps )")
plt.xlabel("iteration")
plt.ylabel("MSE")
plt.tight_layout()
plt.show()

# -----
# 6) Generate ONE reverse trajectory + store per-step reverse Gauss
# -----
@torch.no_grad()
def generate_reverse_chain_with_params():
    """
    Returns arrays for animation:
    t_list:    timesteps (T-1 ... 0) length T
    x_t_list:  current x_t for each step (length T)

```

```

    mu_list:    mu_theta(x_t,t) (length T)
    var_list:   sigma_t^2 (we use DDPM posterior variance tilde_be
    x_prev_list: sampled x_{t-1} produced by sampling N(mu, var)
    ....
# start from x_T ~ N(0,1)
x = torch.randn(1, device=device) # this is x_T
x_t_list, mu_list, var_list, x_prev_list, t_list = [], [], [],

for t_idx in reversed(range(T)):
    t = torch.tensor([t_idx], device=device, dtype=torch.long)
    eps_pred = net(x, t)
    mu = p_mean_from_eps(x, t_idx, eps_pred)

    if t_idx == 0:
        var = torch.tensor(0.0, device=device)
        x_prev = mu # deterministic final step
    else:
        var = posterior_variance(t_idx)
        x_prev = mu + torch.sqrt(var) * torch.randn_like(mu)

    t_list.append(t_idx)
    x_t_list.append(float(x.item()))
    mu_list.append(float(mu.item()))
    var_list.append(float(var.item()))
    x_prev_list.append(float(x_prev.item()))

    x = x_prev # move to next

return (np.array(t_list), np.array(x_t_list), np.array(mu_list)
        np.array(var_list), np.array(x_prev_list))

t_list, x_t_list, mu_list, var_list, x_prev_list = generate_reverse

# -----
# 7) Animate the reverse conditional distributions N(mu_theta, var)
# -----
grid = np.linspace(-6, 6, 700)

def normal_pdf(x, mu, var):
    var = max(var, 1e-12)
    return (1.0 / np.sqrt(2*np.pi*var)) * np.exp(-(x-mu)**2 / (2*va

# robust y-limit based on peaks of Gaussians we will plot
peaks = 1.0 / np.sqrt(2*np.pi*np.maximum(var_list, 1e-12))
ymax = 1.15 * np.percentile(peaks, 95)
ymax = max(ymax, 0.6)

fig, ax = plt.subplots(figsize=(9,4))

```

```

fig.subplots_adjust(bottom=0.28)

(line_pdf,) = ax.plot([], [], lw=2, label=r"$p_{\theta}(x_{t-1} | x_{t-2})$")
dot_xt = ax.scatter([], [], s=70, label=r"current $x_t$")
dot_mu = ax.scatter([], [], s=70, label=r"mean $\mu_{\theta}(x_t, t)$")
dot_xprev = ax.scatter([], [], s=90, label=r"sampled $x_{t-1}$")

ax.set_xlim(grid.min(), grid.max())
ax.set_ylim(0, ymax)
ax.set_xlabel("x")
ax.set_ylabel("density")
ax.set_title("Learned reverse step: conditional Gaussian distribution")
ax.legend(loc="upper right")

status = fig.text(0.02, 0.02, "", ha="left", va="bottom", fontsize=10)

def init():
    line_pdf.set_data([], [])
    dot_xt.set_offsets(np.empty((0,2)))
    dot_mu.set_offsets(np.empty((0,2)))
    dot_xprev.set_offsets(np.empty((0,2)))
    status.set_text("")
    return line_pdf, dot_xt, dot_mu, dot_xprev, status

def update(i):
    # i indexes steps: 0..T-1 corresponds to t = T-1 down to 0
    t_idx = int(t_list[i])
    xt = float(x_t_list[i])
    mu = float(mu_list[i])
    var = float(var_list[i])
    xprev = float(x_prev_list[i])

    pdf = normal_pdf(grid, mu, var if t_idx > 0 else 1e-6) # avoid
    line_pdf.set_data(grid, pdf)

    y0 = 0.02 * ymax
    dot_xt.set_offsets(np.array([[xt, y0]]))
    dot_mu.set_offsets(np.array([[mu, y0]]))
    dot_xprev.set_offsets(np.array([[xprev, y0]]))

    snr = snr_t(t_idx)
    status.set_text(
        f"Reverse step: t={t_idx:3d} -> t-1\n"
        f"current $x_t = \{xt:+.4f\}$\n"
        f"$\mu_{\theta}(x_t, t) = \{mu:+.4f\}$\n"
        f"$\sigma_t^2$ (used) = $\{var:.6f\}$\n"
        f"sampled $x_{t-1} = \{xprev:+.4f\}$\n"
        f"$SNR(t) = \{snr:.4g\}$ ($\{snr_db(snr):+.2f\}$ dB)"
    )

```

```

)

# Update title with t
ax.set_title(f"Learned reverse distribution at step t={t_idx}:")
return line_pdf, dot_xt, dot_mu, dot_xprev, status

anim = FuncAnimation(fig, update, frames=np.arange(len(t_list)), in

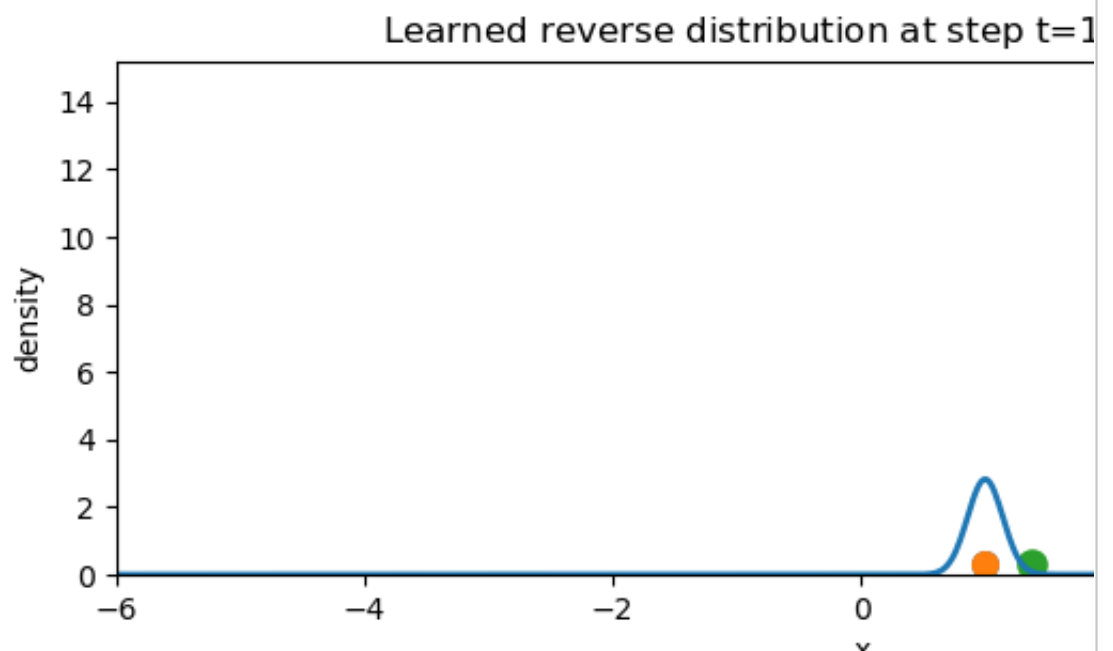
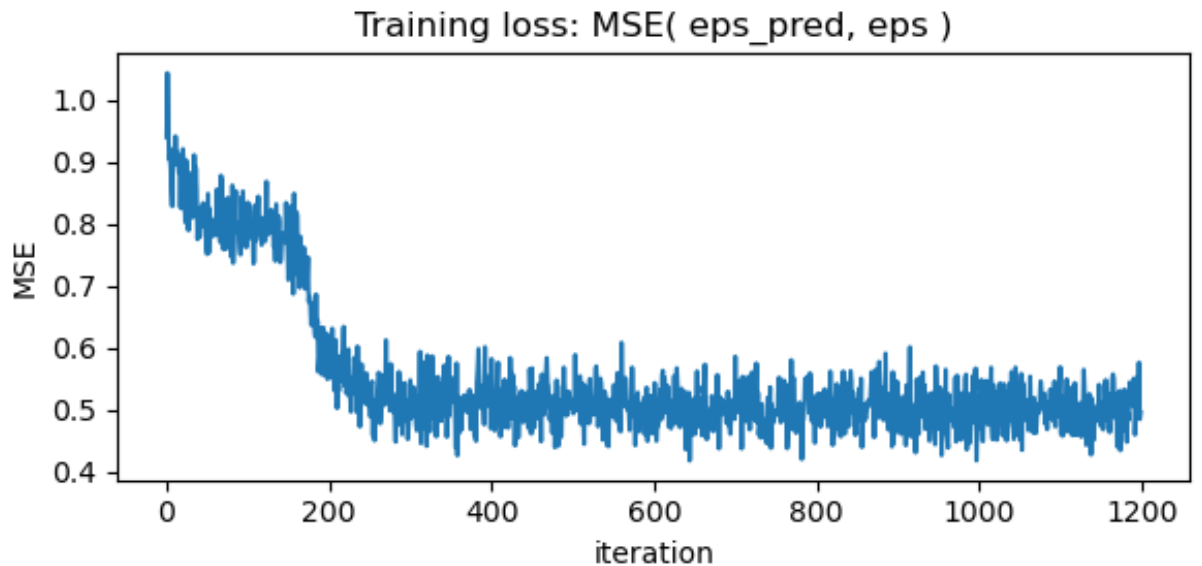
plt.close(fig)
HTML(anim.to_jshtml())

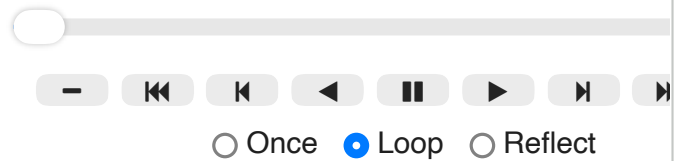
```

```

device: cpu
iter 300/1200 | loss 0.4987
iter 600/1200 | loss 0.5080
iter 900/1200 | loss 0.5462
iter 1200/1200 | loss 0.4965

```





```
import math
import torch
import torch.nn.functional as F

class DiffusionSchedule:
    # Owns betas/alphas/alpha_bars on the correct device.
    def __init__(self, T=200, beta_start=1e-4, beta_end=0.02, device=None):
        self.device = device if device is not None else torch.device('cpu')
        self.T = int(T)
        self.betas = torch.linspace(beta_start, beta_end, self.T, device=self.device)
        self.alphas = 1.0 - self.betas
        self.alpha_bars = torch.cumprod(self.alphas, dim=0)

    def extract(self, a, t, x_shape):
        # a: (T,), t: (B,) int64 => broadcastable tensor shaped like (B, T)
        out = a.gather(0, t) # (B,)
        return out.view(-1, *[1] * (len(x_shape) - 1)))

class DiffusionBase:
    # Base diffusion utilities for epsilon-prediction DDPM/DDIM.
    def __init__(self, eps_model, schedule: DiffusionSchedule):
        self.eps_model = eps_model
        self.sched = schedule

    def q_sample(self, x0, t, noise=None):
        if noise is None:
            noise = torch.randn_like(x0)
        a_bar = self.sched.extract(self.sched.alpha_bars, t, x0.shape)
        xt = torch.sqrt(a_bar) * x0 + torch.sqrt(1.0 - a_bar) * noise
        return xt, noise

    @torch.no_grad()
    def predict_x0(self, xt, t):
        a_bar = self.sched.extract(self.sched.alpha_bars, t, xt.shape)
        eps = self.eps_model(xt, t)
        x0_hat = (xt - torch.sqrt(1.0 - a_bar) * eps) / torch.sqrt(a_bar)
        return x0_hat
```



```

@torch.no_grad()
def ddpm_step(self, xt, t):
    beta_t = self.sched.extract(self.sched.betas, t, xt.shape)
    alpha_t = self.sched.extract(self.sched.alphas, t, xt.shape)
    a_bar_t = self.sched.extract(self.sched.alpha_bars, t, xt.shape)

    eps = self.eps_model(xt, t)
    mean = (1.0 / torch.sqrt(alpha_t)) * (xt - (beta_t / torch.sqrt(alpha_t)) * eps)

    if (t == 0).all():
        return mean
    z = torch.randn_like(xt)
    return mean + torch.sqrt(beta_t) * z

@torch.no_grad()
def ddim_step(self, xt, t, eta=0.0):
    a_bar_t = self.sched.extract(self.sched.alpha_bars, t, xt.shape)
    eps = self.eps_model(xt, t)
    x0_hat = (xt - torch.sqrt(1.0 - a_bar_t) * eps) / torch.sqrt(a_bar_t)

    t_prev = torch.clamp(t - 1, min=0)
    a_bar_prev = self.sched.extract(self.sched.alpha_bars, t_prev, xt.shape)

    eps_small = 1e-12
    frac = (1.0 - a_bar_prev) / (1.0 - a_bar_t + eps_small)
    inside = 1.0 - (a_bar_t / (a_bar_prev + eps_small))
    sigma_t = eta * torch.sqrt(frac * torch.clamp(inside, min=0))

    dir_coeff = torch.sqrt(torch.clamp(1.0 - a_bar_prev - sigma_t**2, min=0))
    mean = torch.sqrt(a_bar_prev) * x0_hat + dir_coeff * eps

    if (t == 0).all():
        return mean
    z = torch.randn_like(xt)
    return mean + sigma_t * z

@torch.no_grad()
def sample(self, n, sampler="ddpm", eta=0.0, stride=1, shape=None):
    if shape is None:
        raise ValueError("Provide shape=(n, ...) for sampling.")
    xt = torch.randn(*shape, device=self.sched.device)
    times = list(range(self.sched.T - 1, -1, -stride))
    for time in times:
        t = torch.full((shape[0],), time, device=self.sched.device)
        if sampler == "ddpm":
            xt = self.ddpm_step(xt, t)
        elif sampler == "ddim":
            xt = self.ddim_step(xt, t, eta=eta)

```

```

        else:
            raise ValueError("sampler must be 'ddpm' or 'ddim'")
        return xt

class Diffusion1D(DiffusionBase):
    # For scalar 1D data (B,1) or 1D signals (B,C,L)
    def train_step(self, x0, optimizer):
        self.eps_model.train()
        B = x0.shape[0]
        t = torch.randint(0, self.sched.T, (B,), device=self.sched.device)
        xt, eps = self.q_sample(x0, t)
        eps_pred = self.eps_model(xt, t)
        loss = F.mse_loss(eps_pred, eps)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        return float(loss.item())

    @torch.no_grad()
    def eval_step(self, x0):
        self.eps_model.eval()
        B = x0.shape[0]
        t = torch.randint(0, self.sched.T, (B,), device=self.sched.device)
        xt, eps = self.q_sample(x0, t)
        eps_pred = self.eps_model(xt, t)
        loss = F.mse_loss(eps_pred, eps)
        return float(loss.item()), x0, xt, eps, eps_pred, t

class Diffusion2D(DiffusionBase):
    # For images (B,C,H,W)
    def train_step(self, x0, optimizer):
        self.eps_model.train()
        B = x0.shape[0]
        t = torch.randint(0, self.sched.T, (B,), device=self.sched.device)
        xt, eps = self.q_sample(x0, t)
        eps_pred = self.eps_model(xt, t)
        loss = F.mse_loss(eps_pred, eps)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        return float(loss.item())

    @torch.no_grad()
    def eval_step(self, x0):
        self.eps_model.eval()
        B = x0.shape[0]
        t = torch.randint(0, self.sched.T, (B,), device=self.sched.device)
        xt, eps = self.q_sample(x0, t)

```

```
eps_pred = self.eps_model(xt, t)
loss = F.mse_loss(eps_pred, eps)
return float(loss.item()), x0, xt, eps, eps_pred, t
```